

Sequence Models

Advanced Machine Learning and Computer Vision

2 October 2026

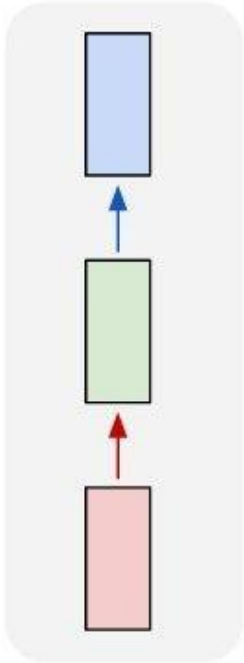
Prof. Fabio Galasso



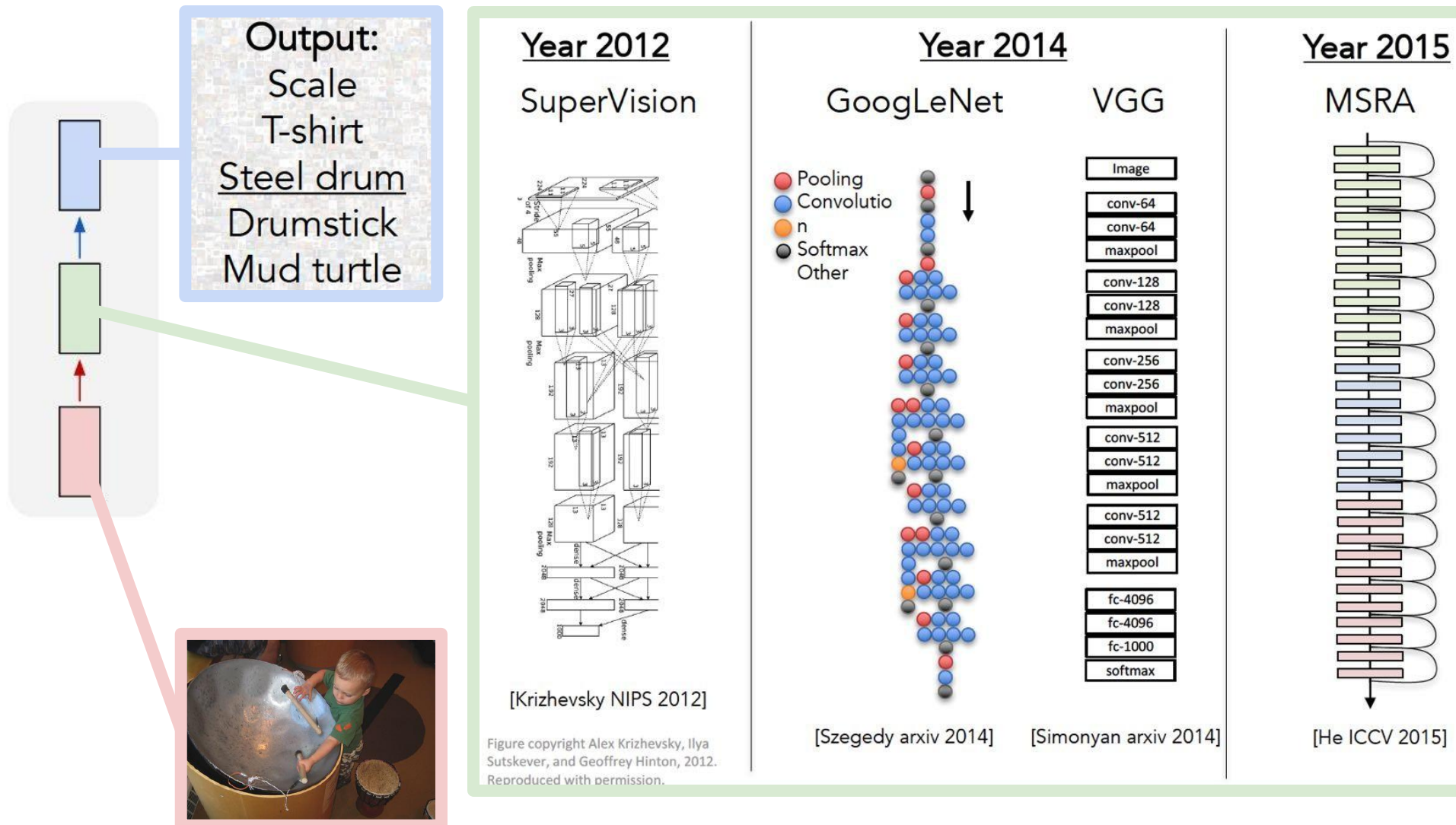
SAPIENZA
UNIVERSITÀ DI ROMA

“Vanilla” Neural Network

one to one

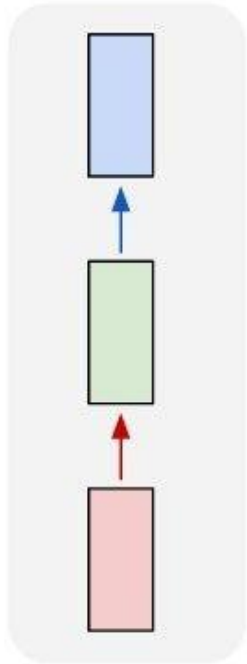


“Vanilla” Neural Network

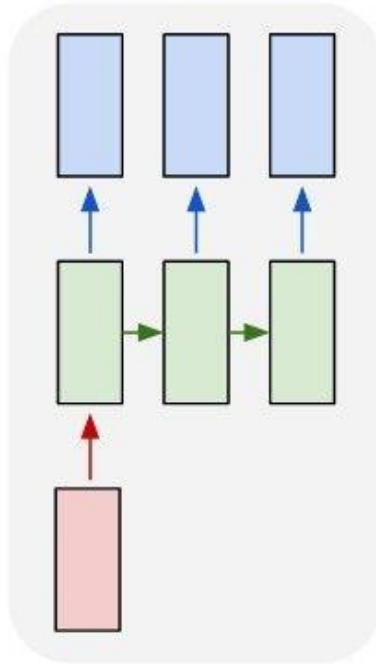


Process Sequences

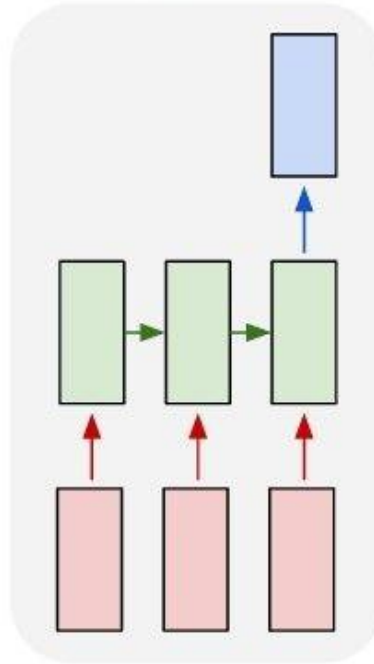
one to one



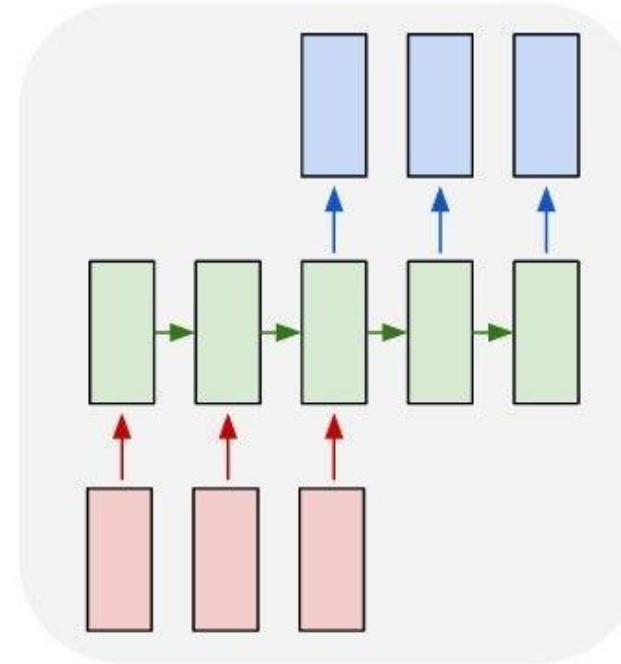
one to many



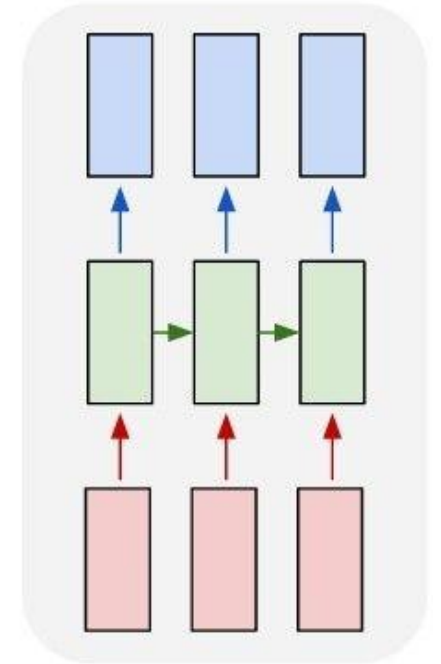
many to one



many to many



many to many



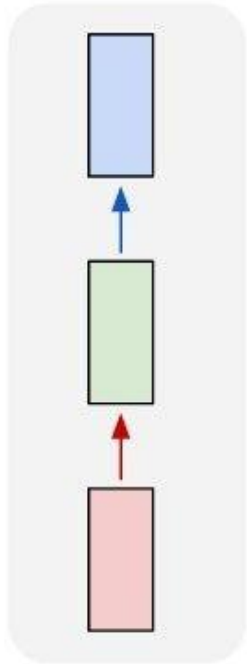
Vanilla Neural Networks

Sequence tasks

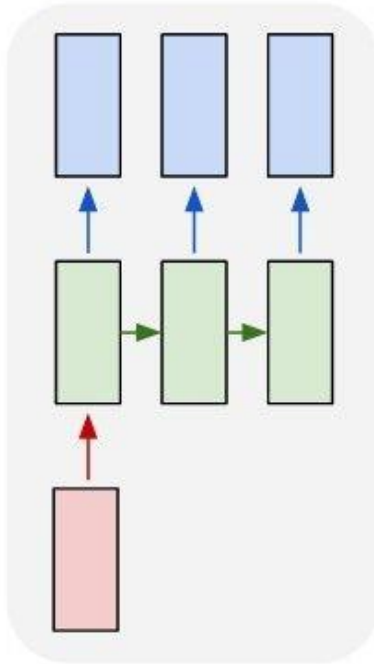
captioning, video classification, forecasting, ...

Process Sequences

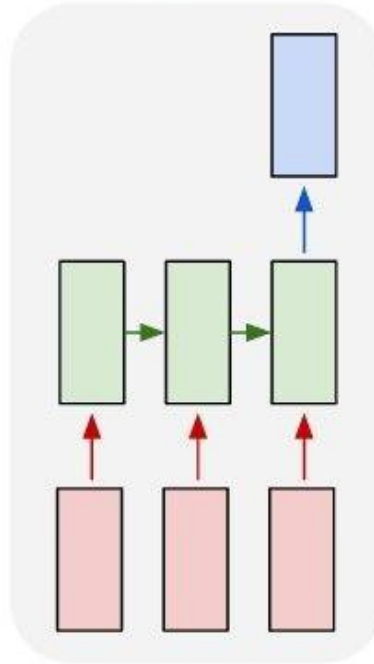
one to one



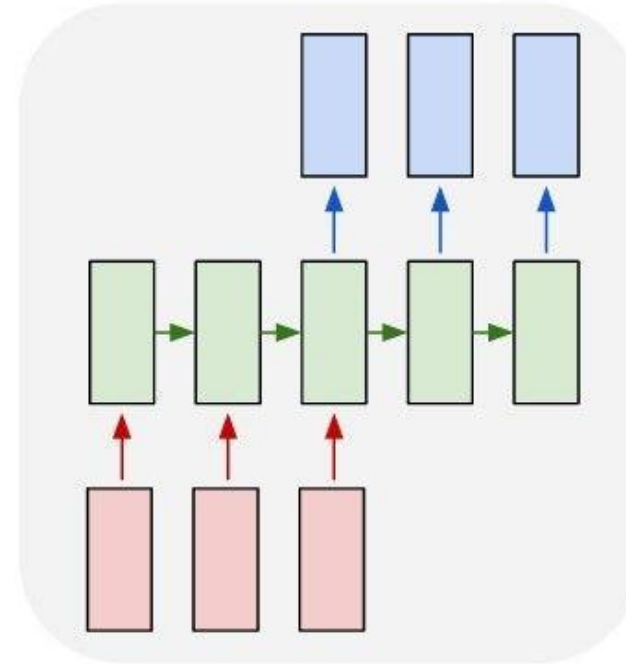
one to many



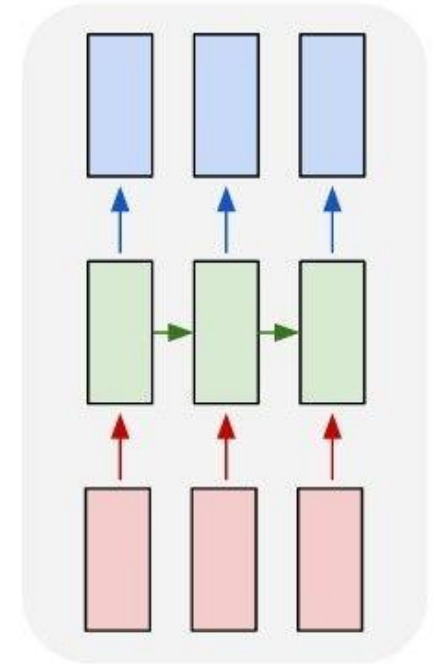
many to one



many to many



many to many



↖ e.g. **Image Captioning**
image -> sequence of words

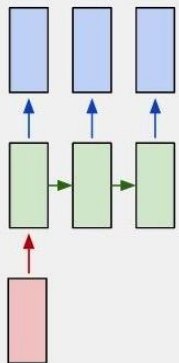
Process Sequences

Sequences in the output...
(one-to-many)



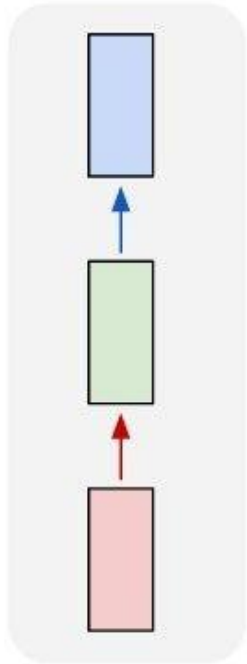
A happy brown dog.

one to many

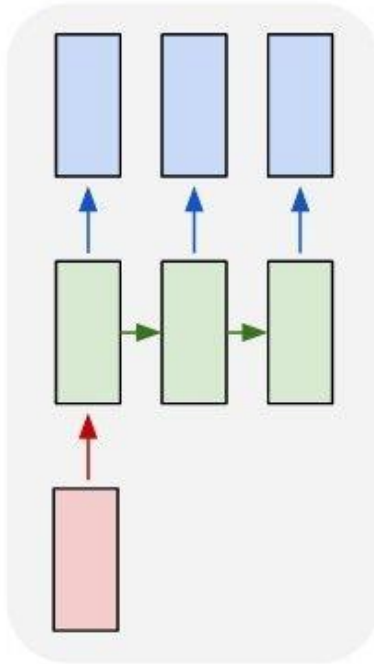


Process Sequences

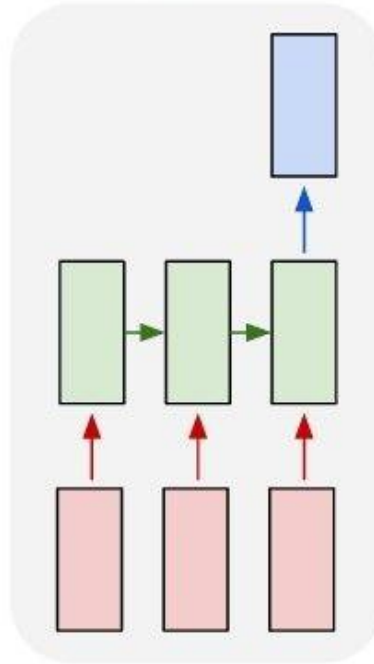
one to one



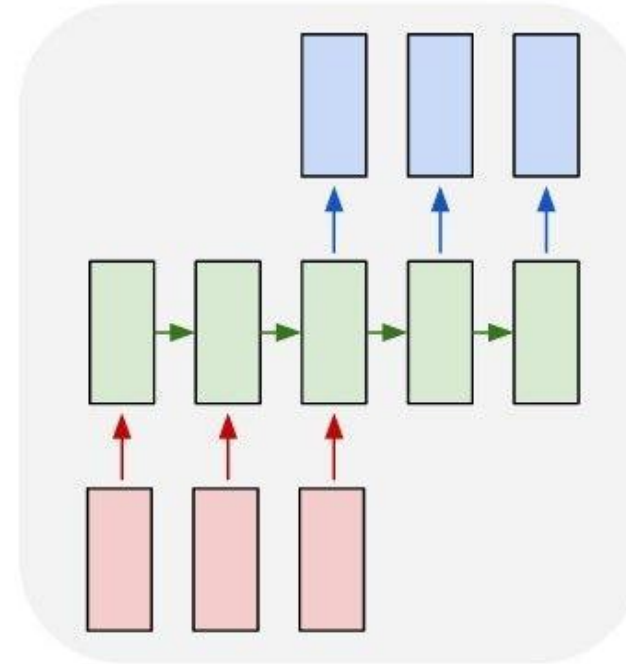
one to many



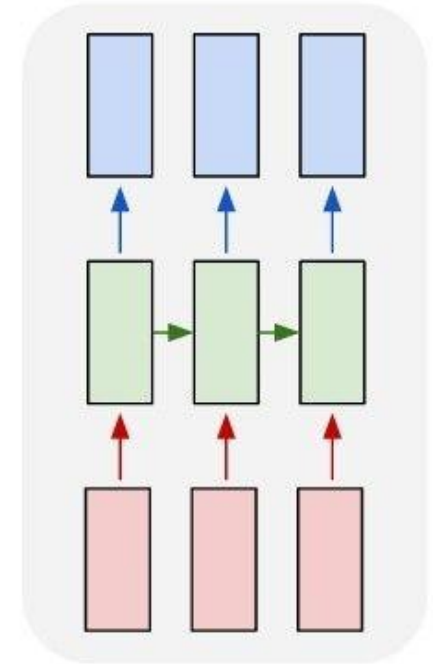
many to one



many to many



many to many



e.g. **action prediction**
sequence of video frames -> action class

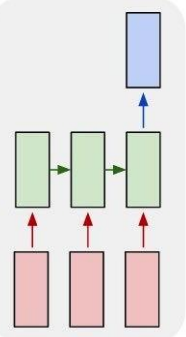
Process Sequences

Sequences in the input...
(many-to-one)



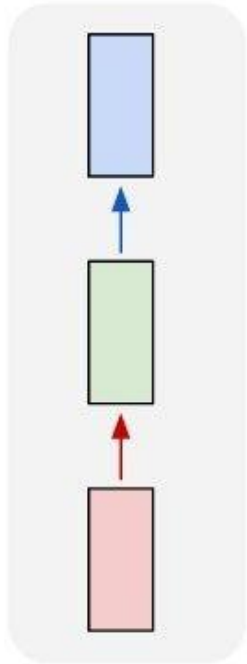
- ➔
- ☐ Running
 - ☒ Jumping
 - ☐ Dancing
 - ☐ Fighting
 - ☐ Eating

many to one

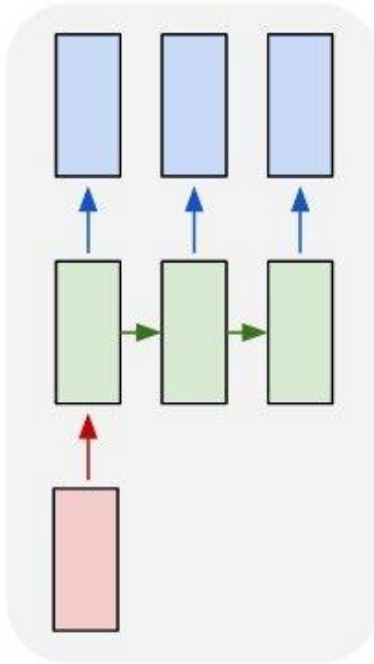


Process Sequences

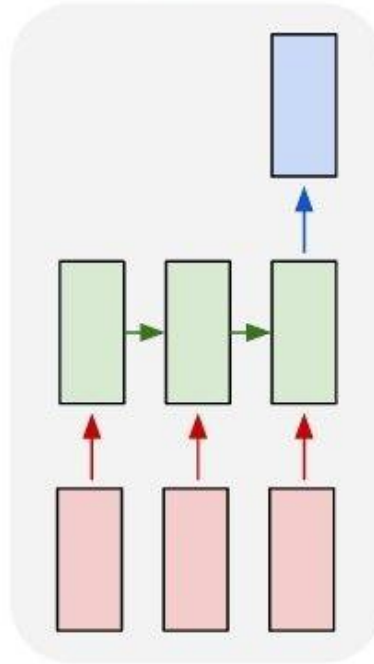
one to one



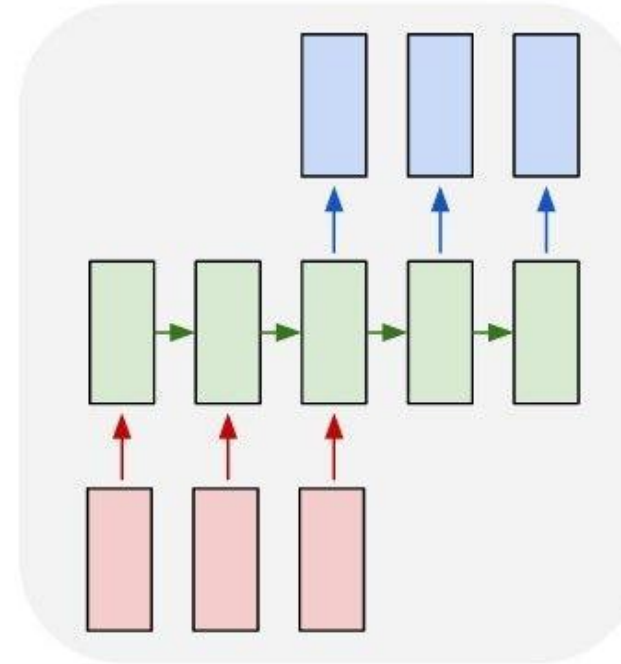
one to many



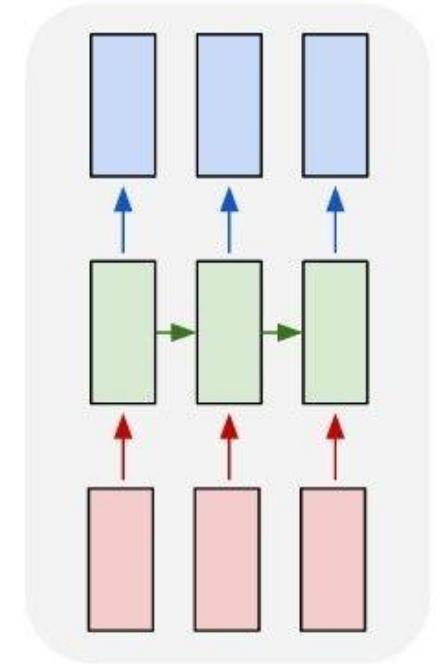
many to one



many to many



many to many



↖ e.g. **Machine Translation**
seq of words -> seq of words

Process Sequences

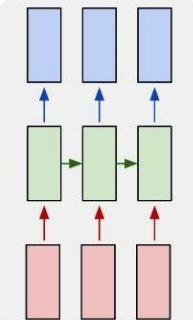
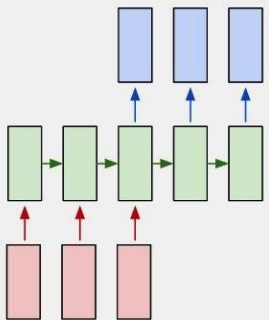
Video Captioning

Sequence of video frames -> caption
(many-to-many)



many to many

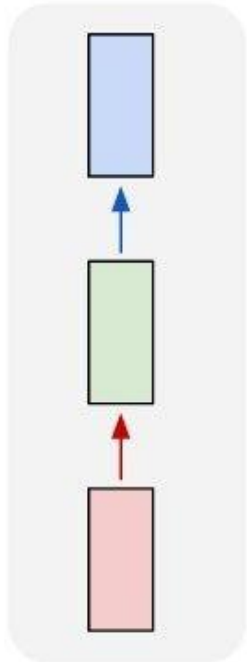
many to many



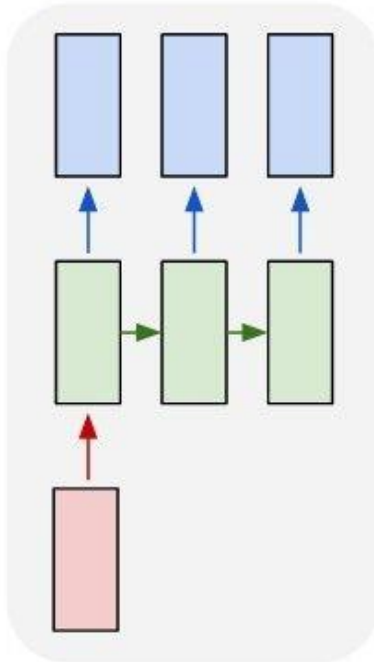
A dog jumps over a hurdle.

Process Sequences

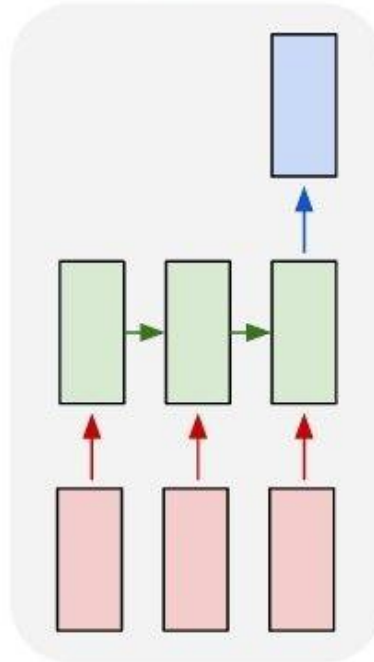
one to one



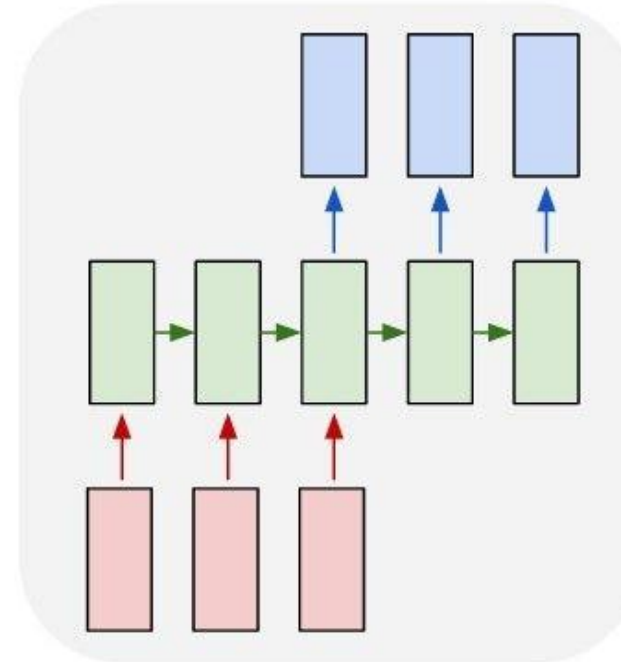
one to many



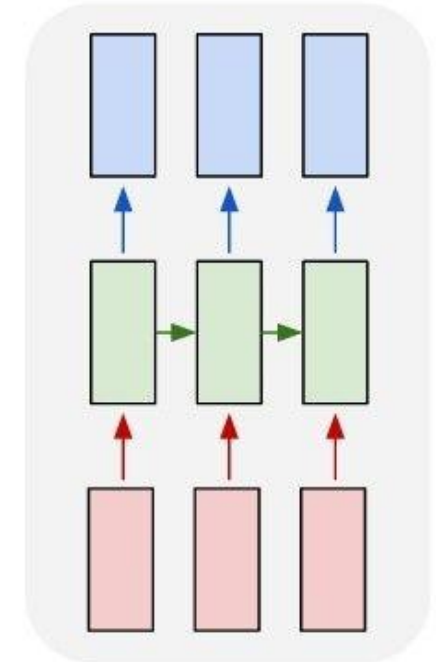
many to one



many to many



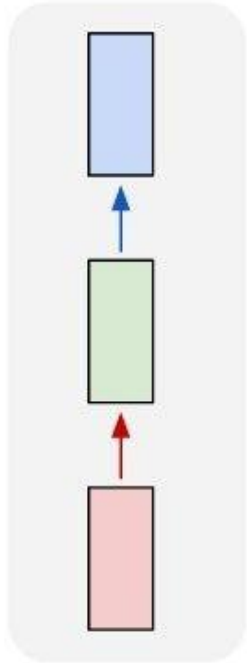
many to many



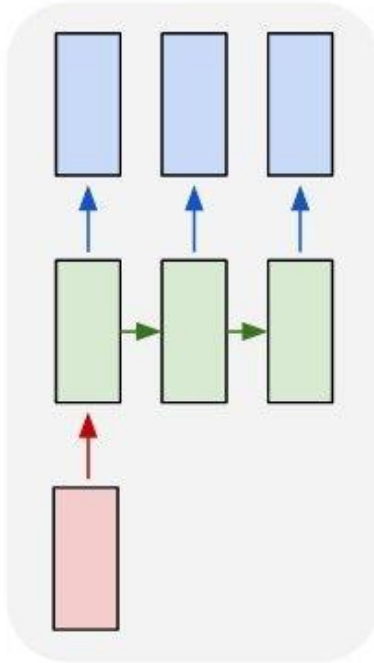
e.g. Video classification on frame level

Process Sequences

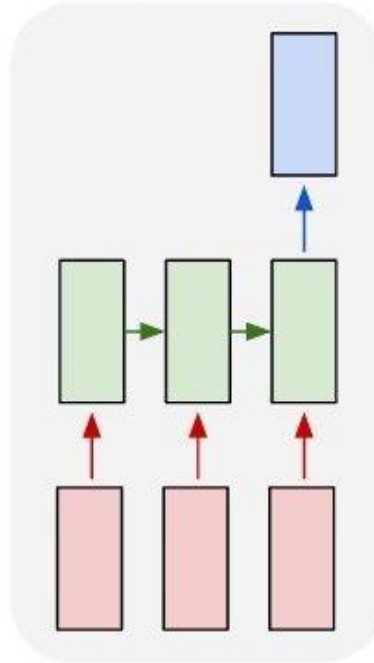
one to one



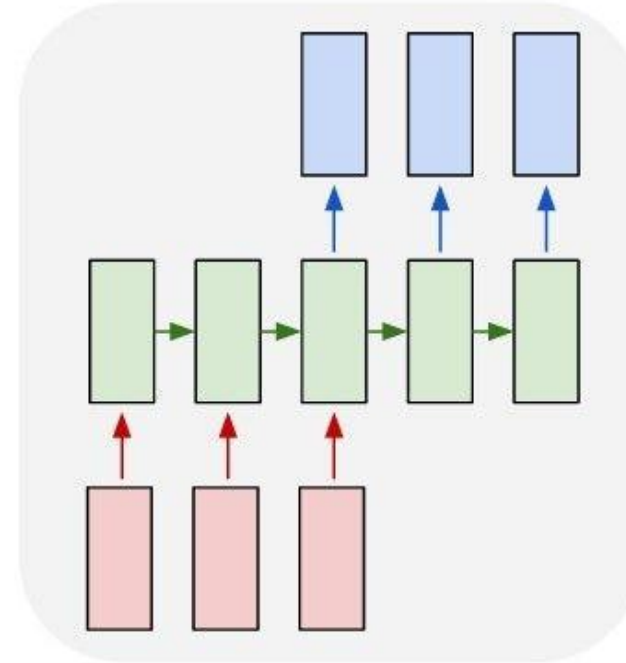
one to many



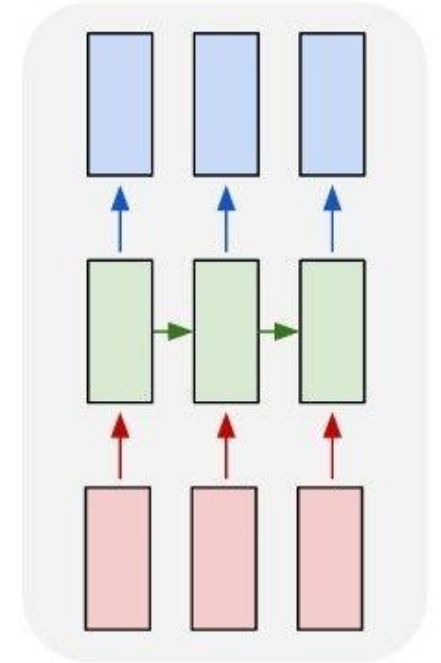
many to one



many to many

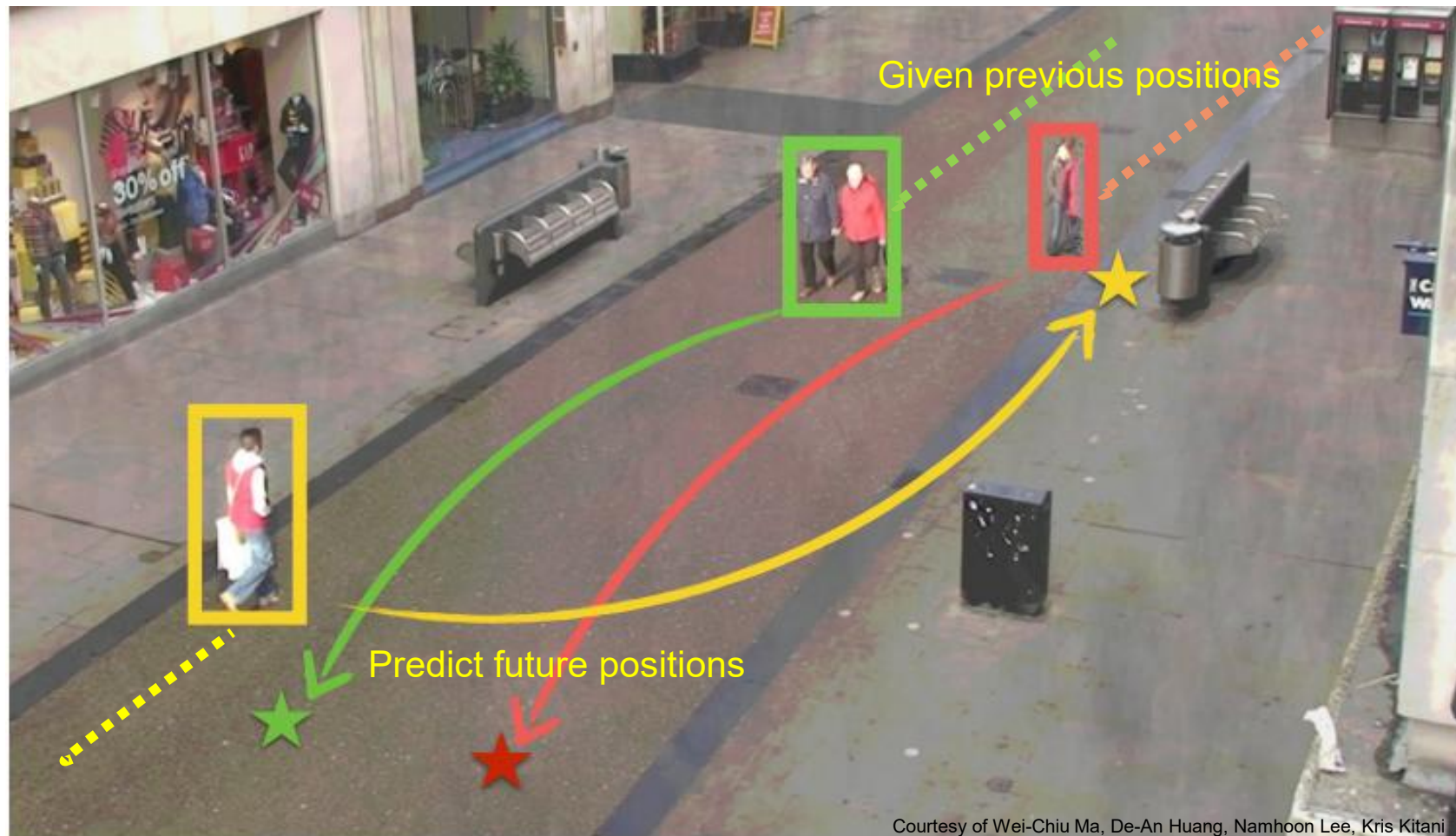


many to many



Forecasting: Given some observations (history)
Predict the future

Trajectory forecasting



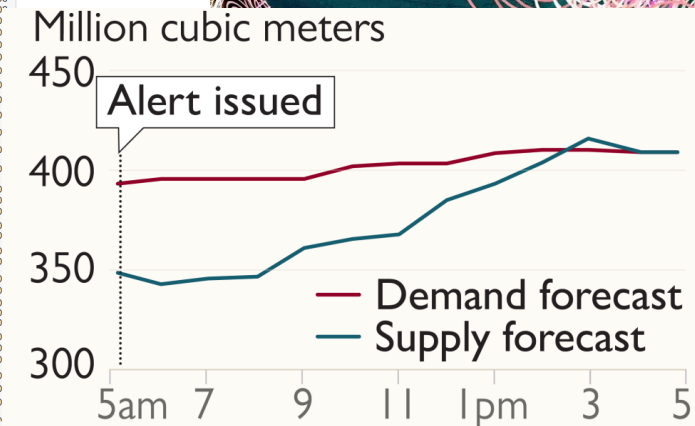
Forecasting

- Human-robot interaction
- Long-term tracking across occlusions
- Safety of autonomous vehicles



Also with Forecasting

- Weather and natural disasters, e.g. earthquakes
- Market trends
- Critical pressure of a power plant
- Gas supply and demand
- ...



Outline

- CNN based techniques
- Recurrent Neural Networks

Outline

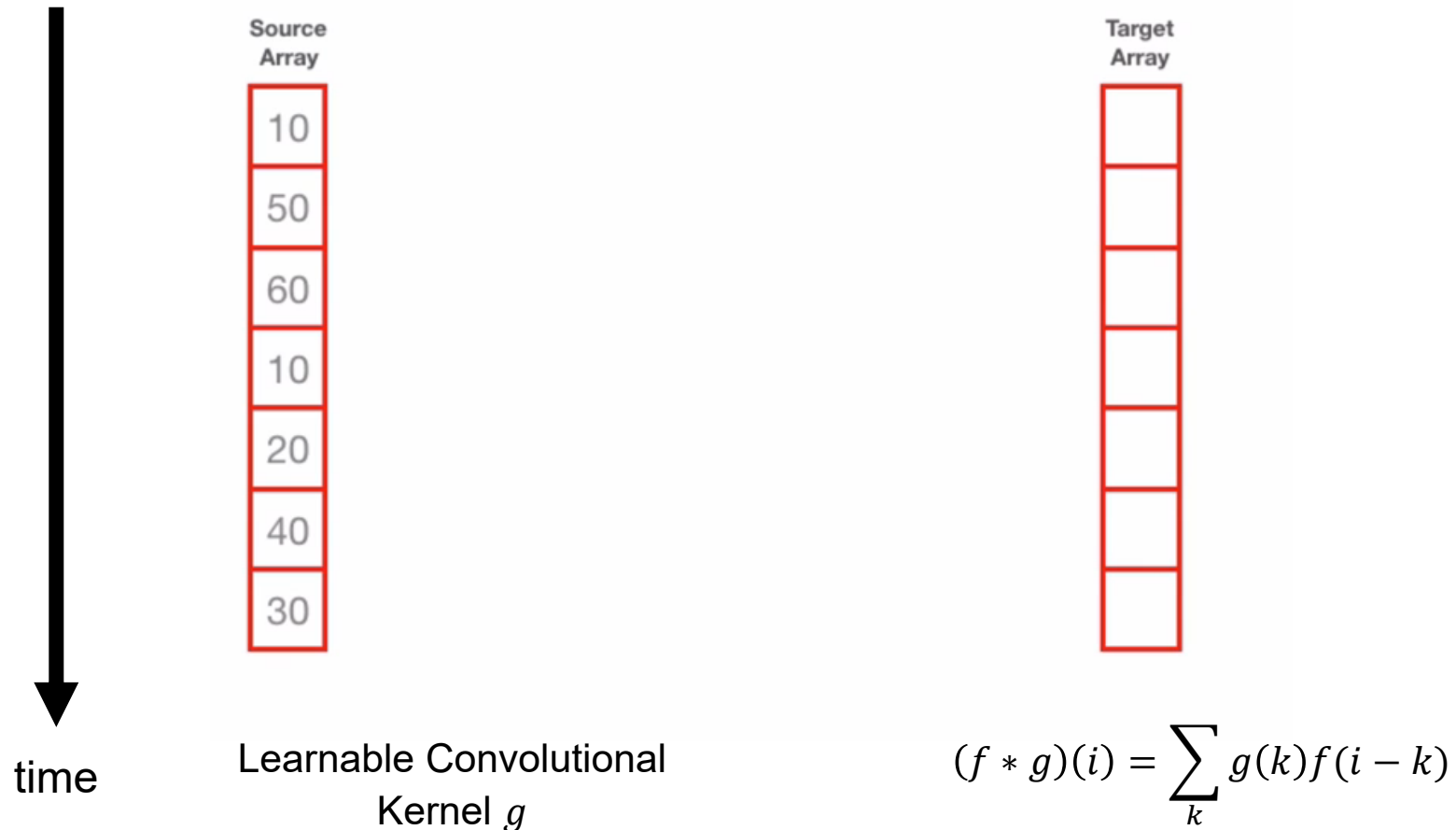
- CNN based techniques
- Recurrent Neural Networks

Temporal Convolution Networks



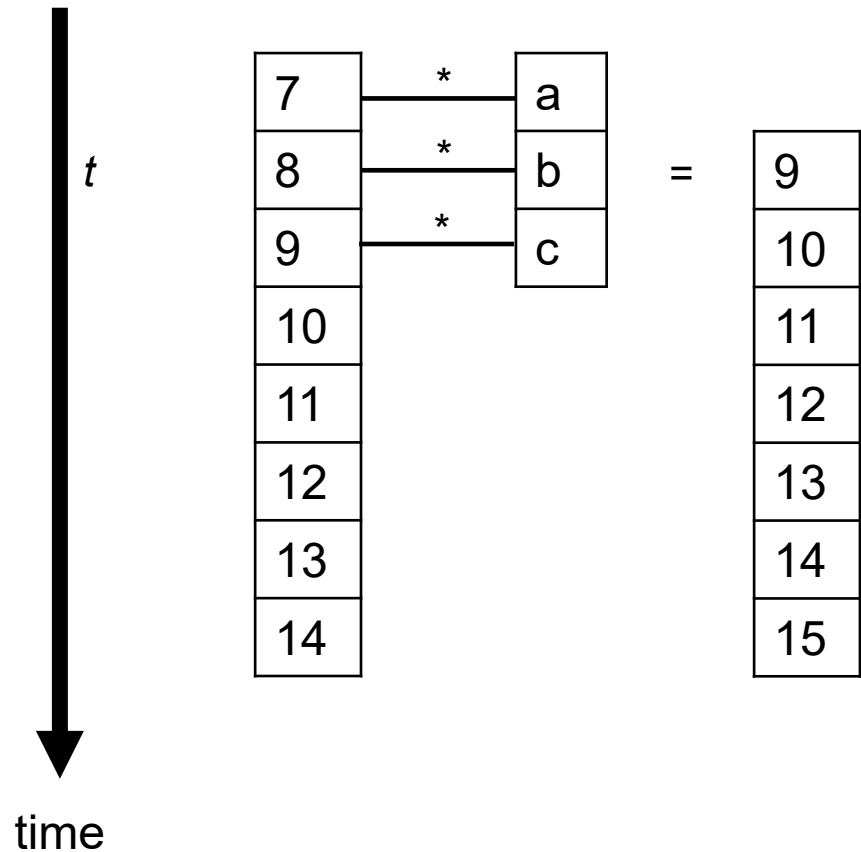
Convolutional Neural Networks

- 1D-convolutions may model sequences



Convolutional Neural Networks

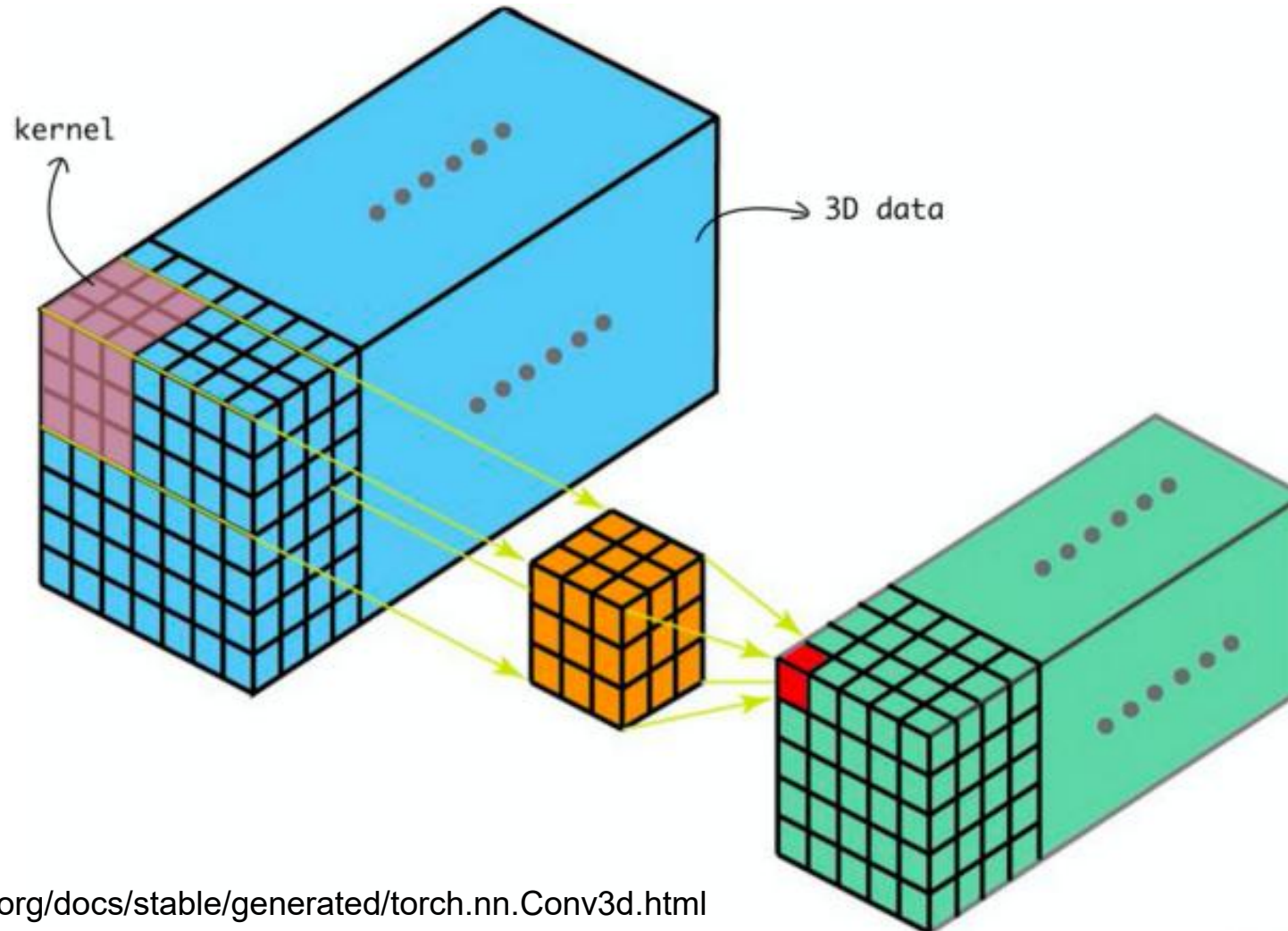
- What 1D convolution kernel of size 3 predicts the future?



Can I leverage longer sequences for a better prediction?

If I consider a larger kernel, what happens if the sequence is shorter?

Applicable to 2D (images) and 3D (volumes, images+time)

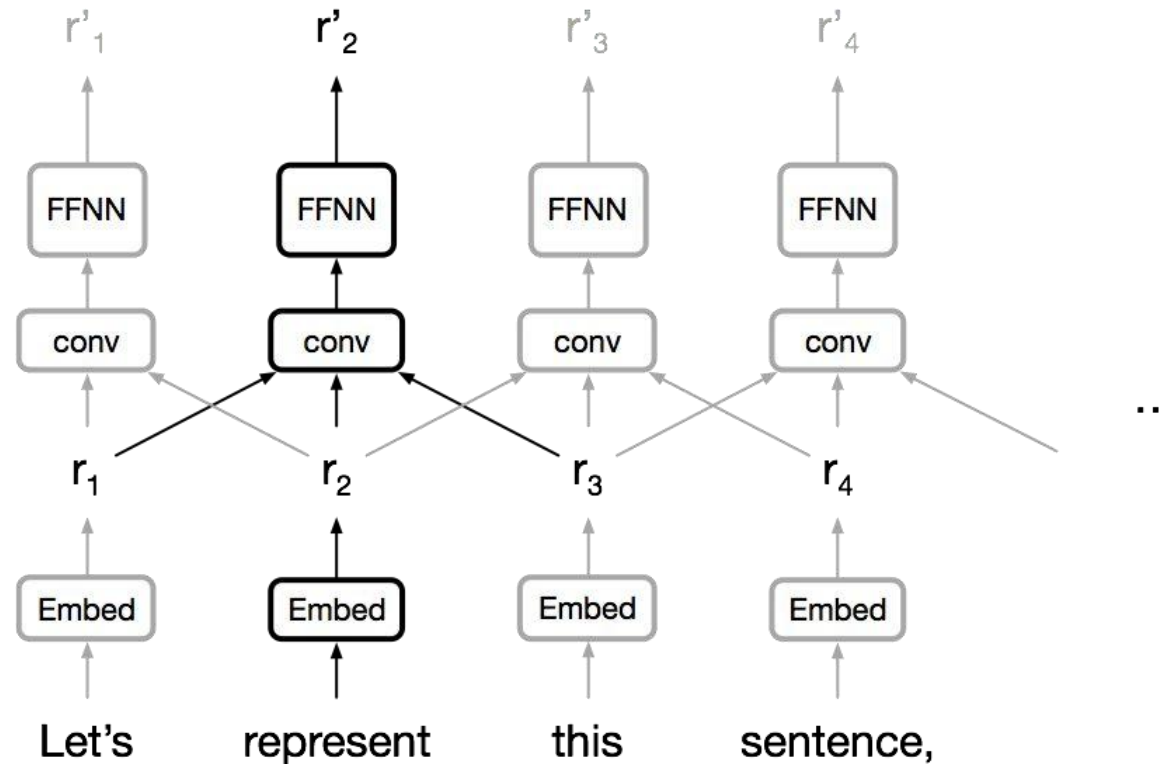


See also:

<https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv3d.html>

Convolutional Neural Networks

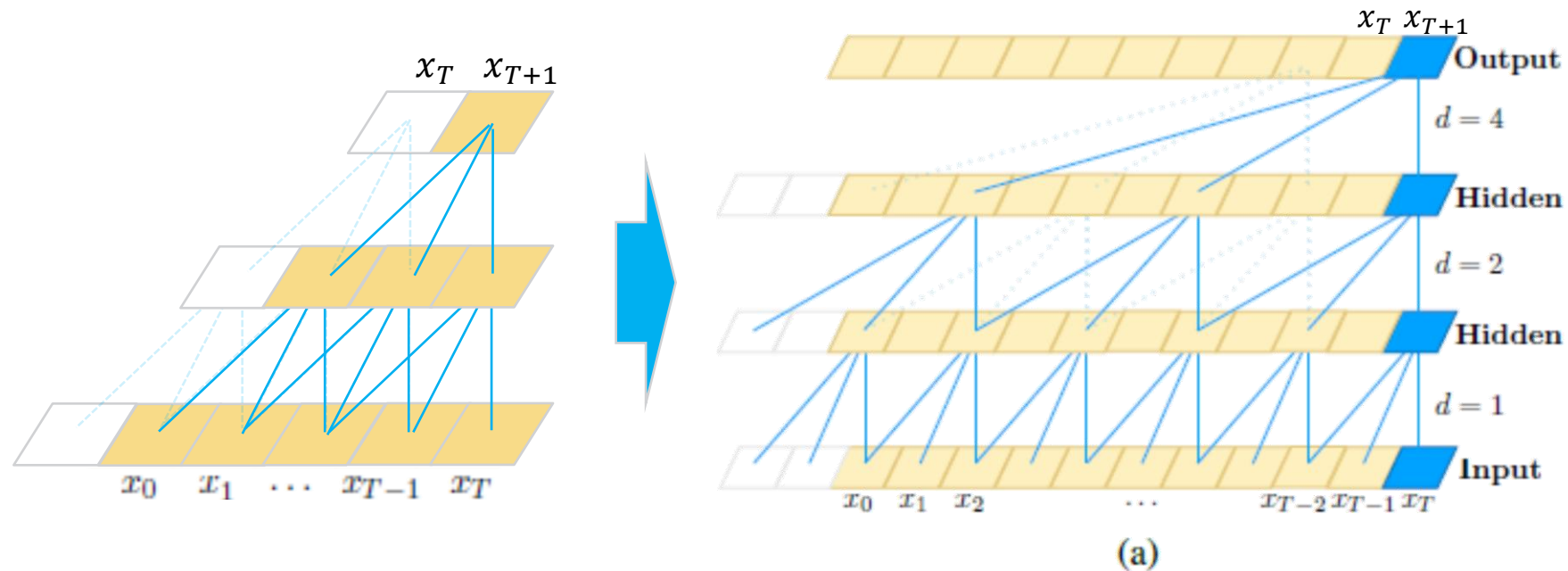
- Also suitable to complex input-output



- But need causal convolutions in forecasting!

Temporal Convolutional Networks (TCN)

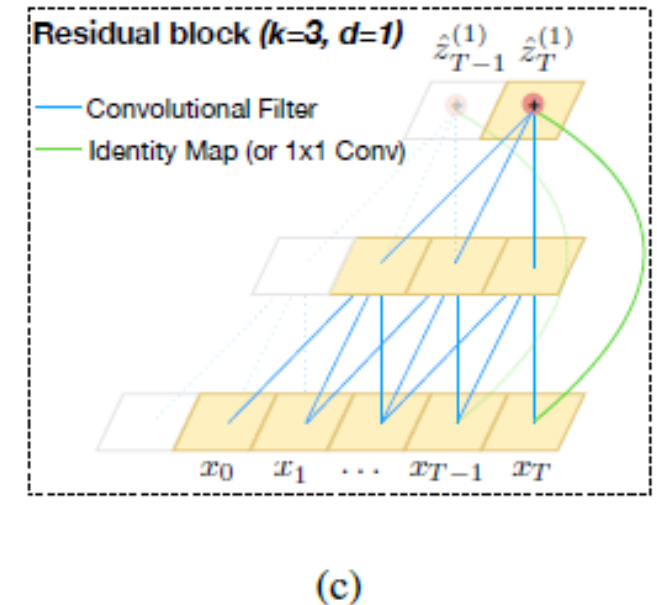
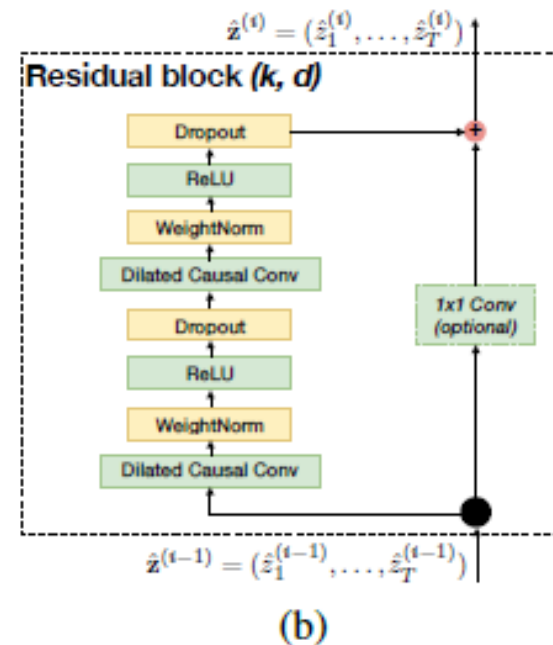
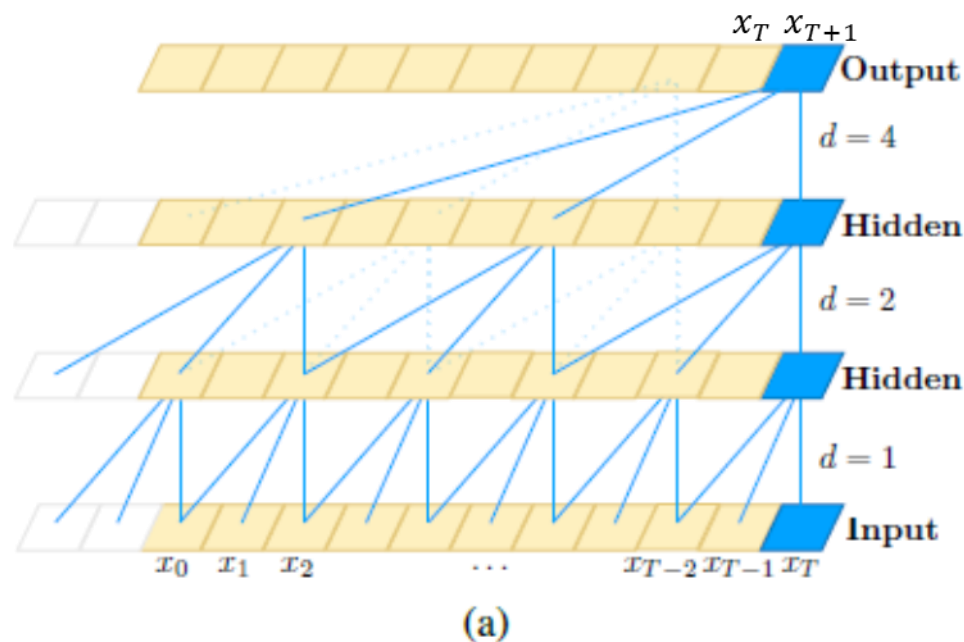
- Trivial to parallelize and exploits local dependencies
- But “interaction distance” is linear in the number of layers
 - Recall: receptive field $1 + L \cdot (K - 1)$, K filter size, L layers $\rightarrow$ long-term dependencies need many layers
- Dilation makes it logarithmic (w/o gaps)



Bai et al. ArXiv'18 “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling”

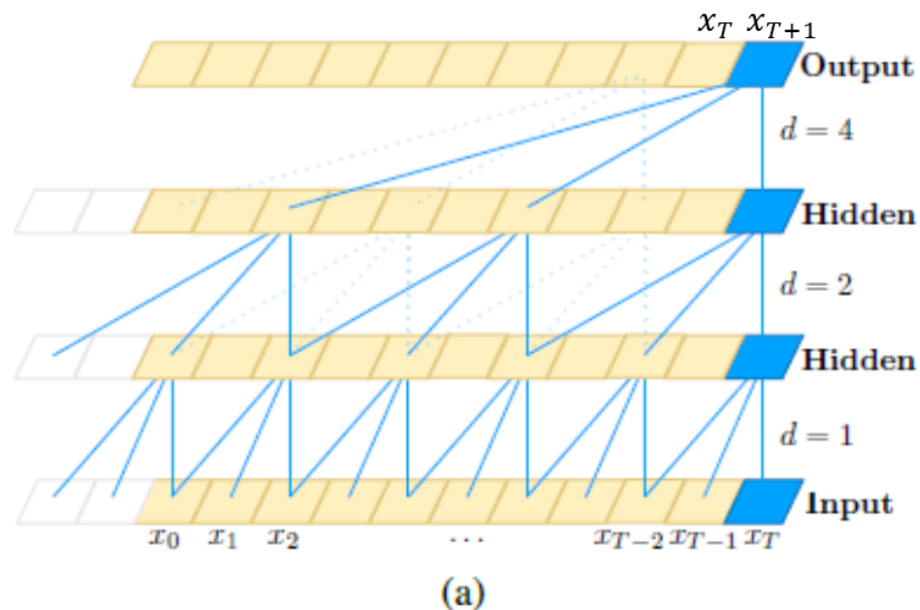
Temporal Convolutional Networks (TCN)

- Bai et al. ArXiv'18 “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling”
 - A few details make the difference, e.g. the residual connection
 - <http://github.com/locuslab/TCN>



Temporal Convolutional Networks (TCN)

- Bai et al. ArXiv'18 “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling”
 - Easy to train and exceeding expectations
 - Needs compromise on flexibility, e.g. variable number of input frames



Sequence Modeling Task	Model Size ($\approx$)	Models			
		LSTM	GRU	RNN	TCN
Seq. MNIST (accuracy ^h)	70K	87.2	96.2	21.5	99.0
Permuted MNIST (accuracy)	70K	85.7	87.3	25.3	97.2
Adding problem $T=600$ (loss ^ℓ)	70K	0.164	5.3e-5	0.177	5.8e-5
Copy memory $T=1000$ (loss)	16K	0.0204	0.0197	0.0202	3.5e-5
Music JSB Chorales (loss)	300K	8.45	8.43	8.91	8.10
Music Nottingham (loss)	1M	3.29	3.46	4.05	3.07
Word-level PTB (perplexity ^ℓ)	13M	78.93	92.48	114.50	89.21
Word-level Wiki-103 (perplexity)	-	48.4	-	-	45.19
Word-level LAMBADA (perplexity)	-	4186	-	14725	1279
Char-level PTB (bpc ^ℓ)	3M	1.41	1.42	1.52	1.35
Char-level text8 (bpc)	5M	1.52	1.56	1.69	1.45

Take-Home Message on TCN

- Pros
 - Trivial to parallelize
 - Exploits local dependencies
- Cons
 - “Interaction distance” between positions linear or logarithmic
 - Long-distance dependencies require many layers

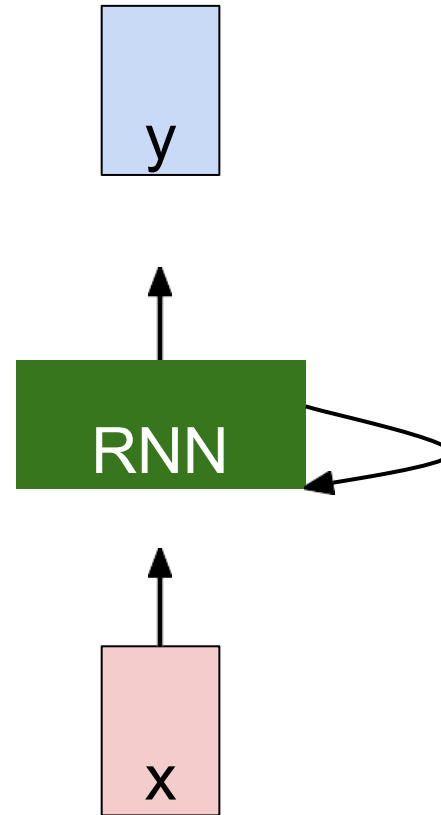
Outline

- CNN based techniques
- Recurrent Neural Networks

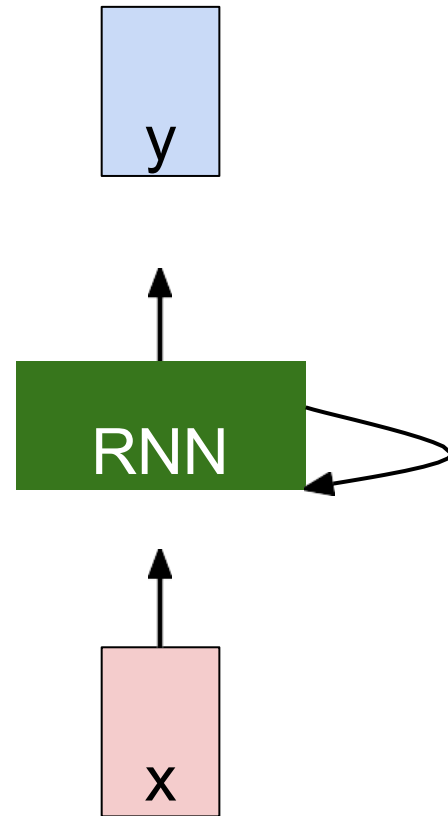
Recurrent Neural Networks



Recurrent Neural Network

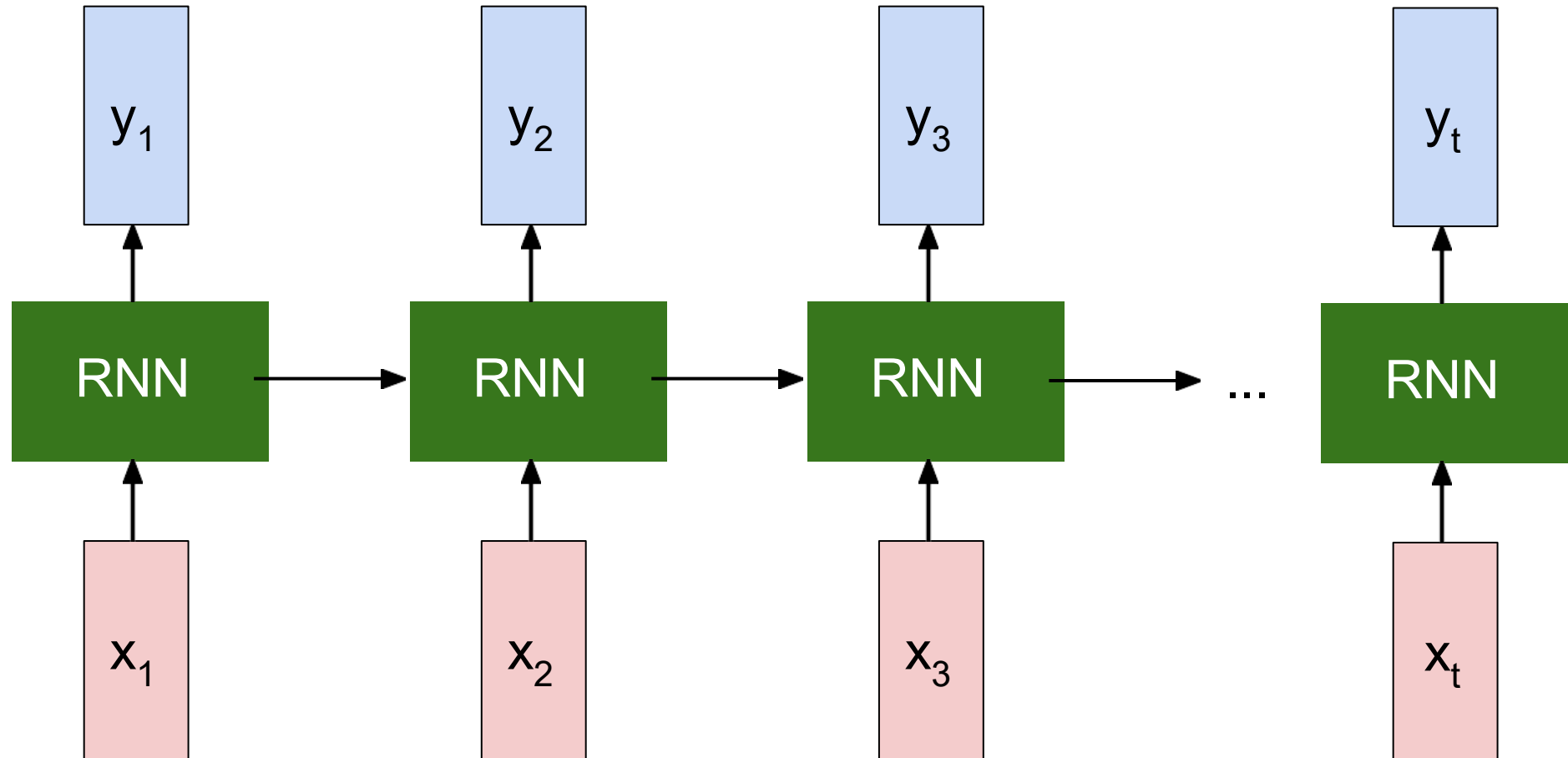


Recurrent Neural Network



Key idea: RNNs have an “internal state” that is updated as a sequence is processed

Unrolled RNN



Recurrent Neural Network

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

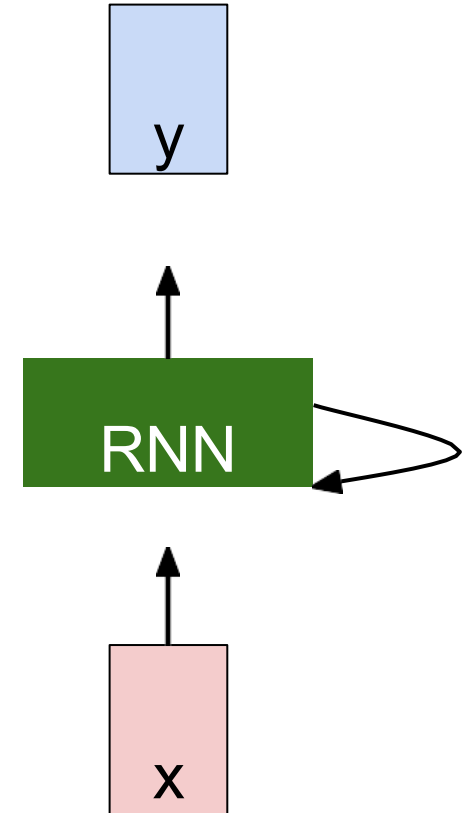
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

some function with parameters W

old state

input vector at some time step



RNN output generation

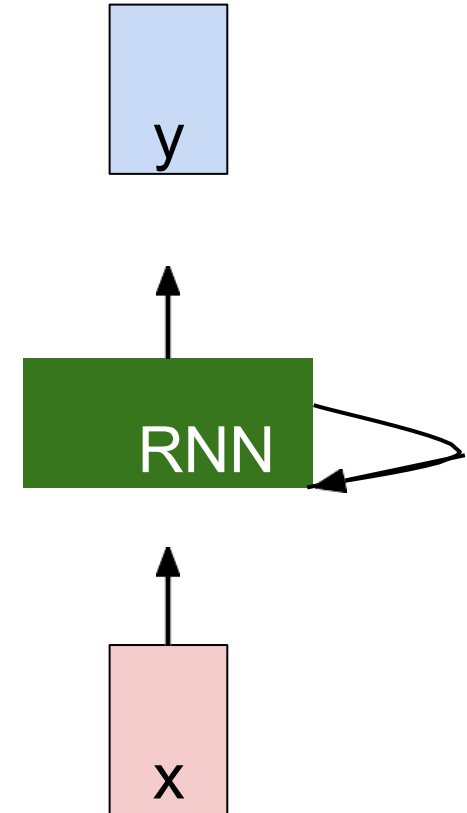
We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$\boxed{y_t} = \boxed{f_{W_{hy}}}(\boxed{h_t})$$

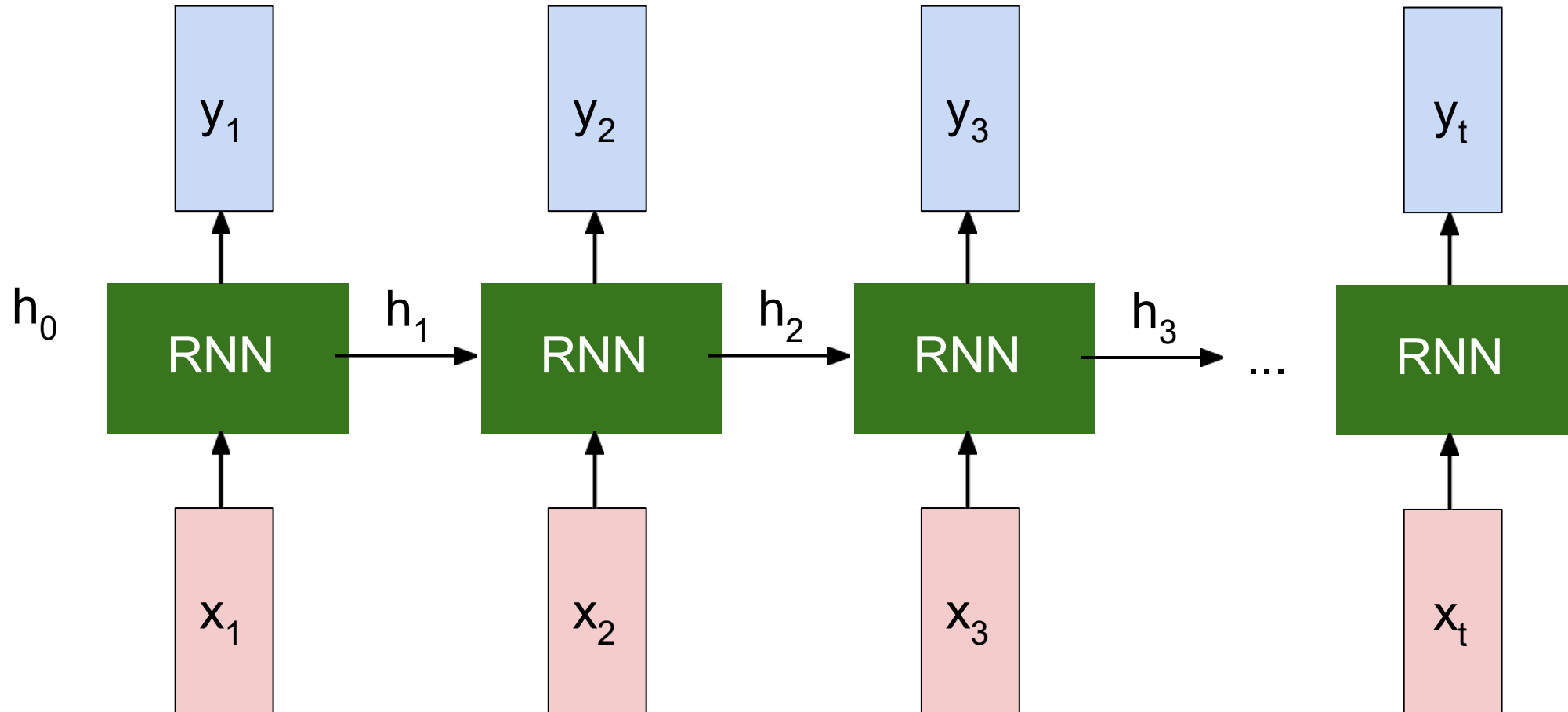
output

another function with parameters W_{hy}

new state



Unrolled RNN

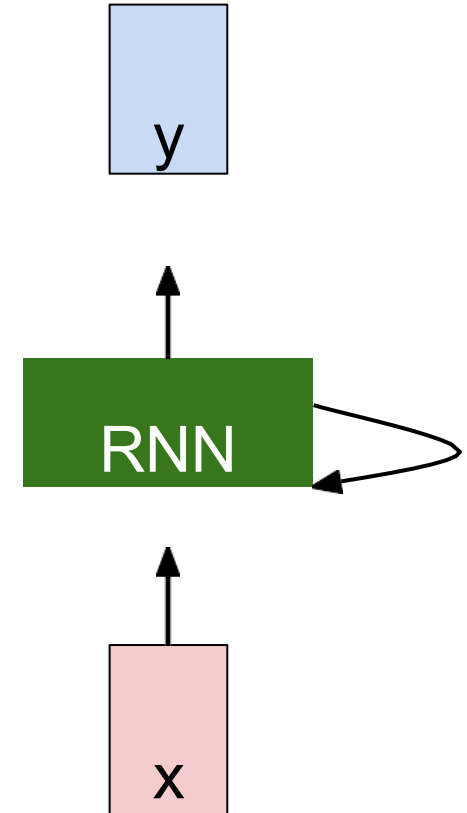


Recurrent Neural Network

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

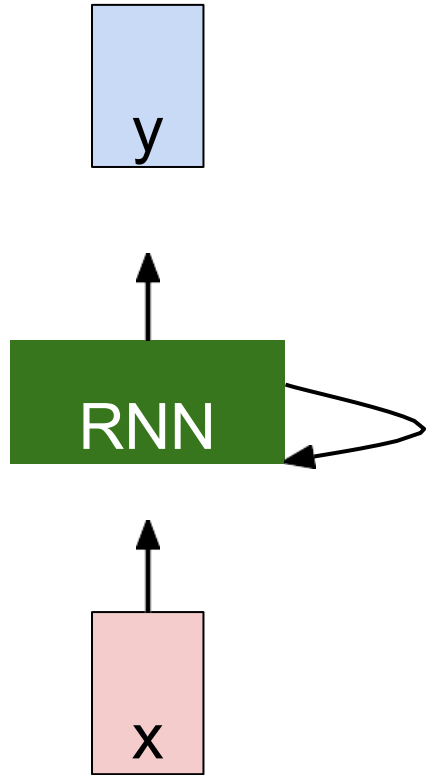
$$h_t = f_W(h_{t-1}, x_t)$$

Notice: the same function and the same set of parameters are used at every time step.



(Vanilla) Recurrent Neural Network

The state consists of a single “*hidden*” vector h :



$$h_t = f_W(h_{t-1}, x_t)$$

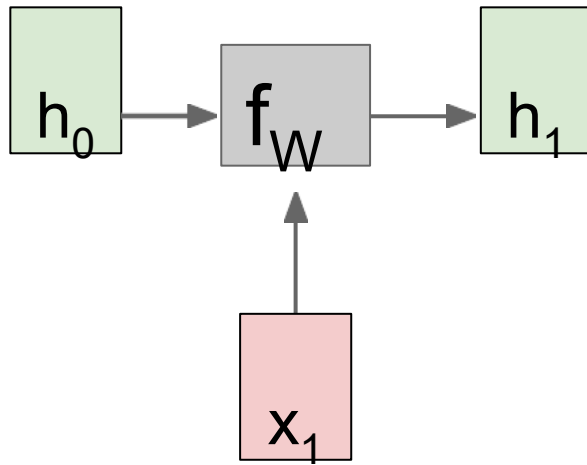


$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

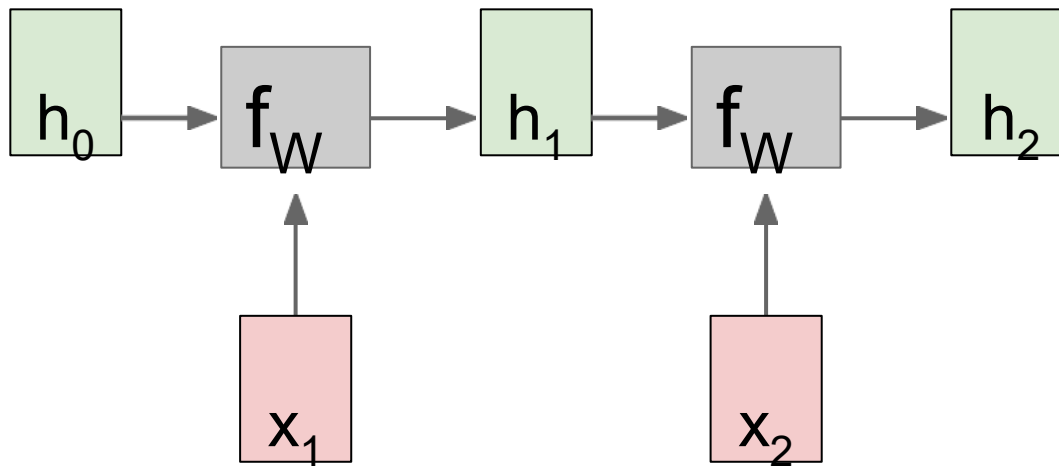
$$y_t = W_{hy}h_t$$

Sometimes called a “Vanilla RNN” or an “Elman RNN” after Prof. Jeffrey Elman

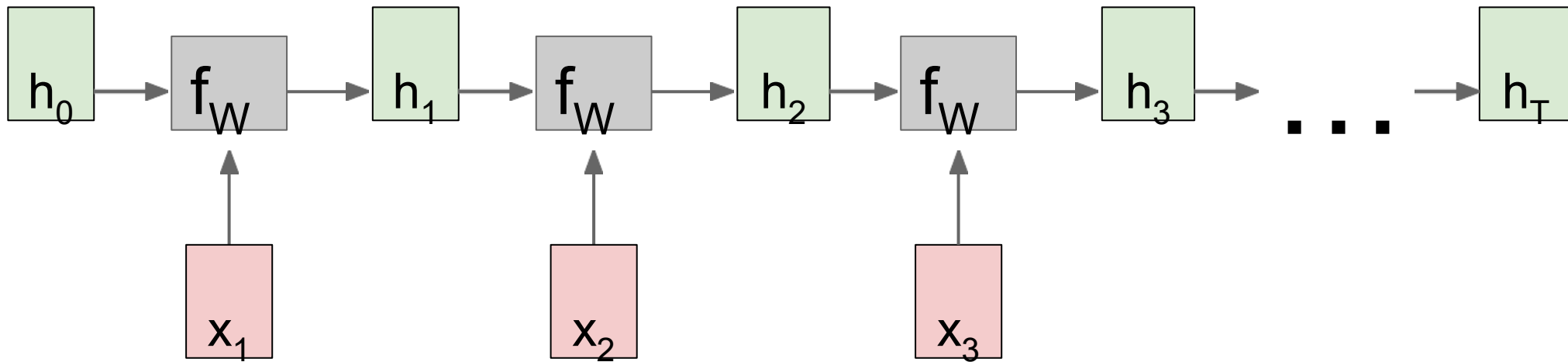
RNN: Computational Graph



RNN: Computational Graph

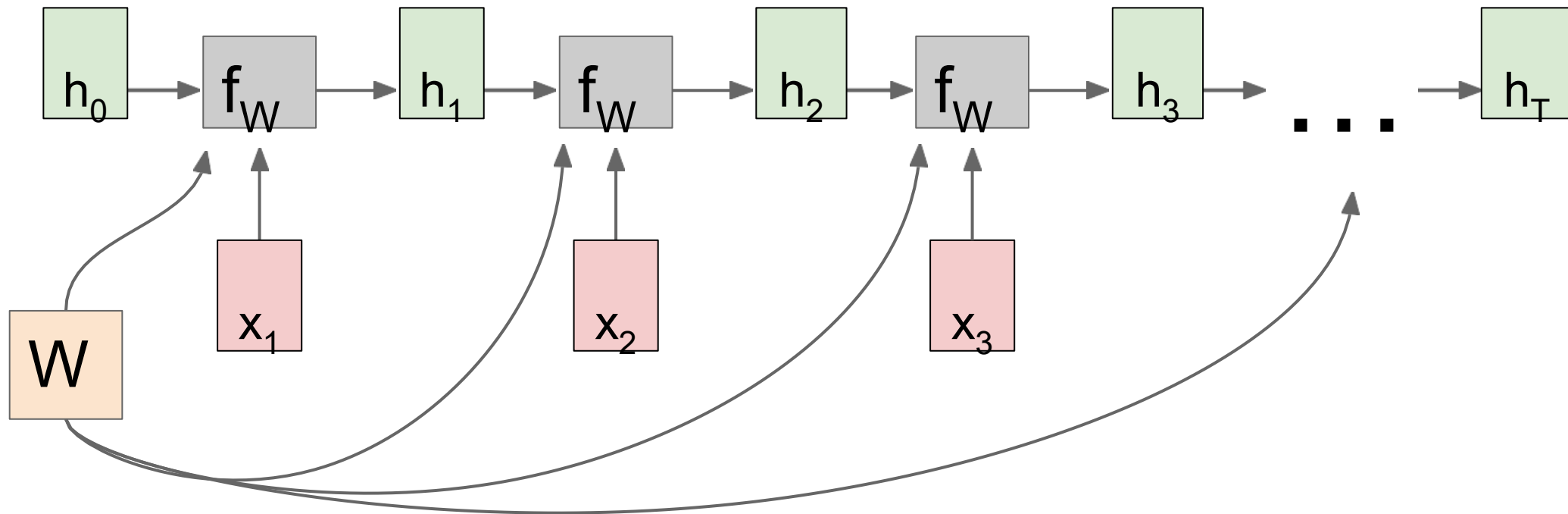


RNN: Computational Graph

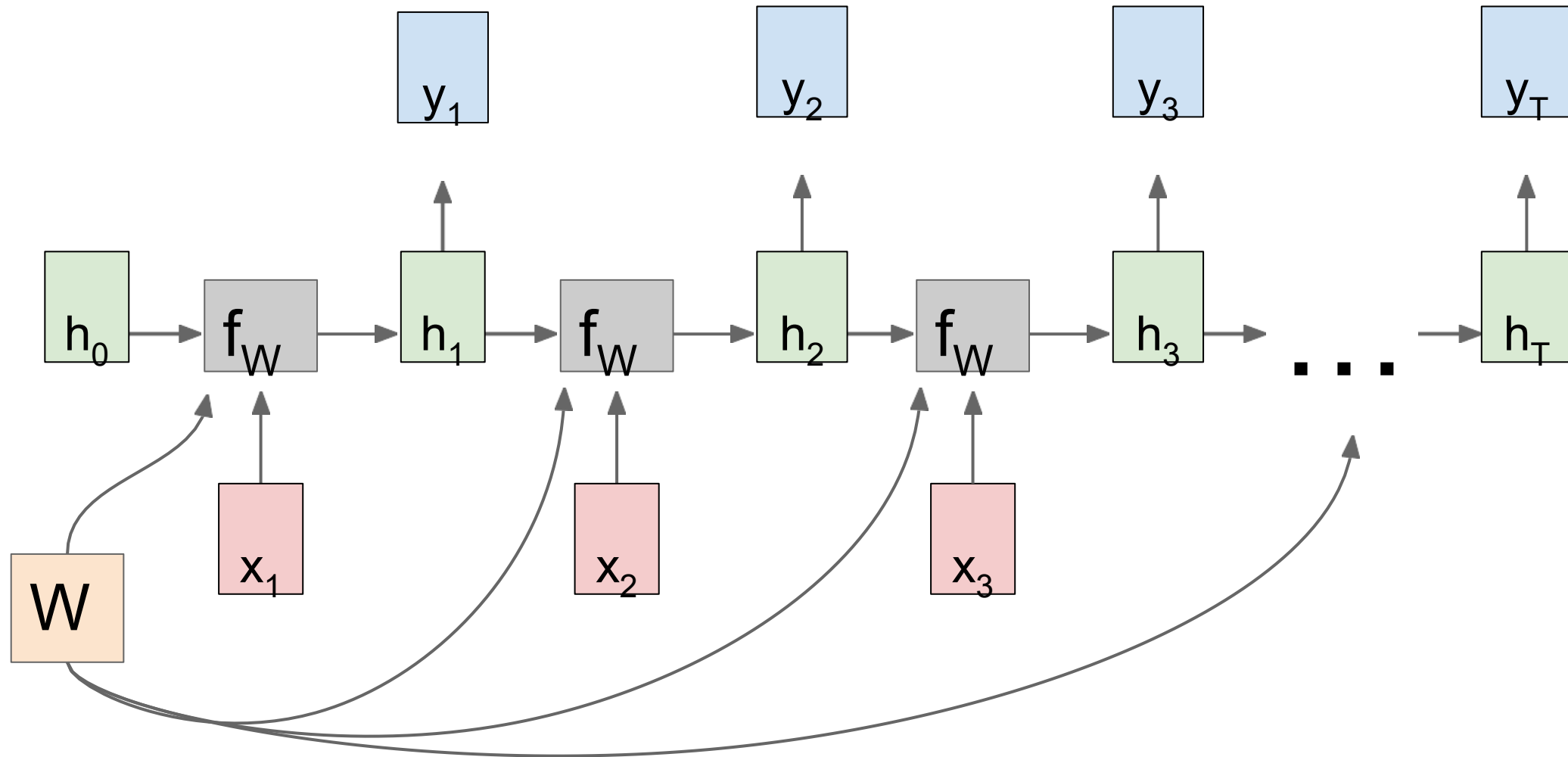


RNN: Computational Graph

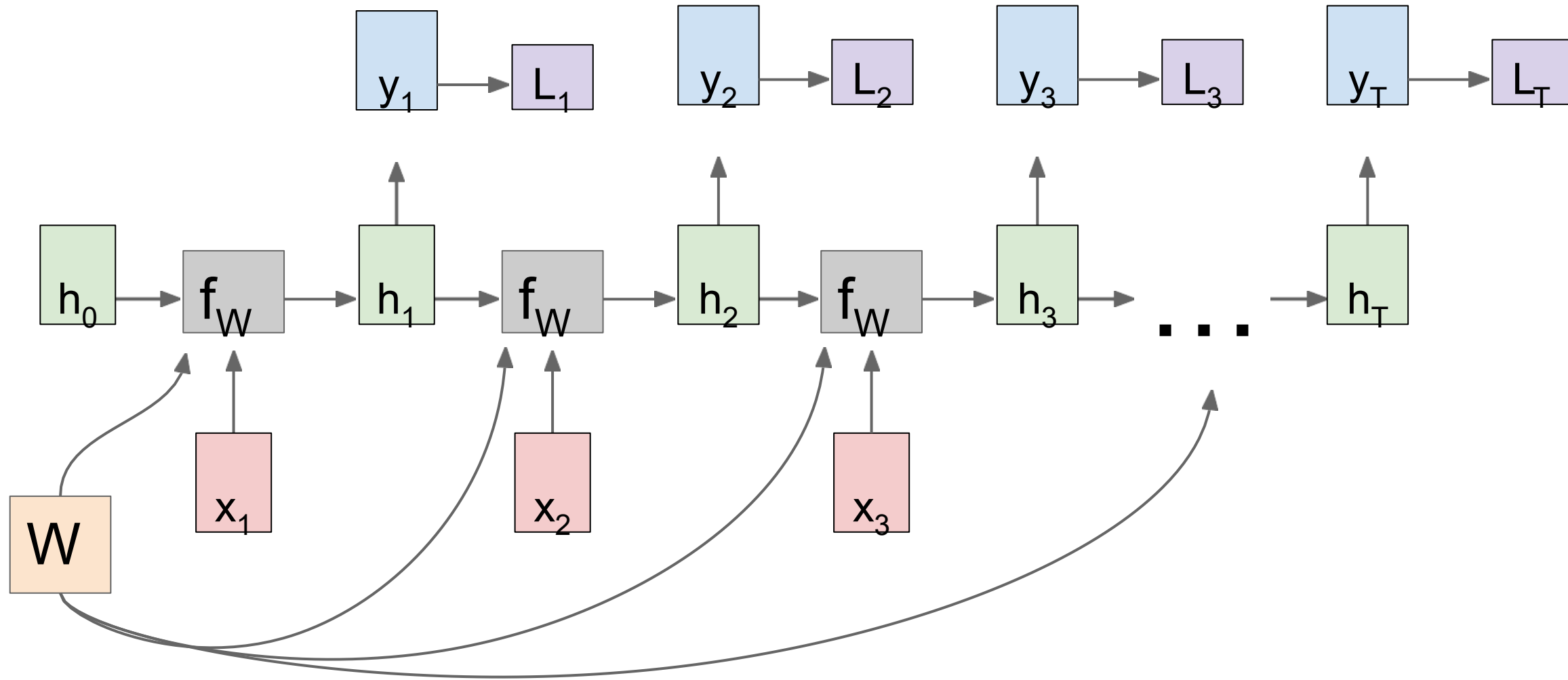
Re-use the same weight matrix at every time-step



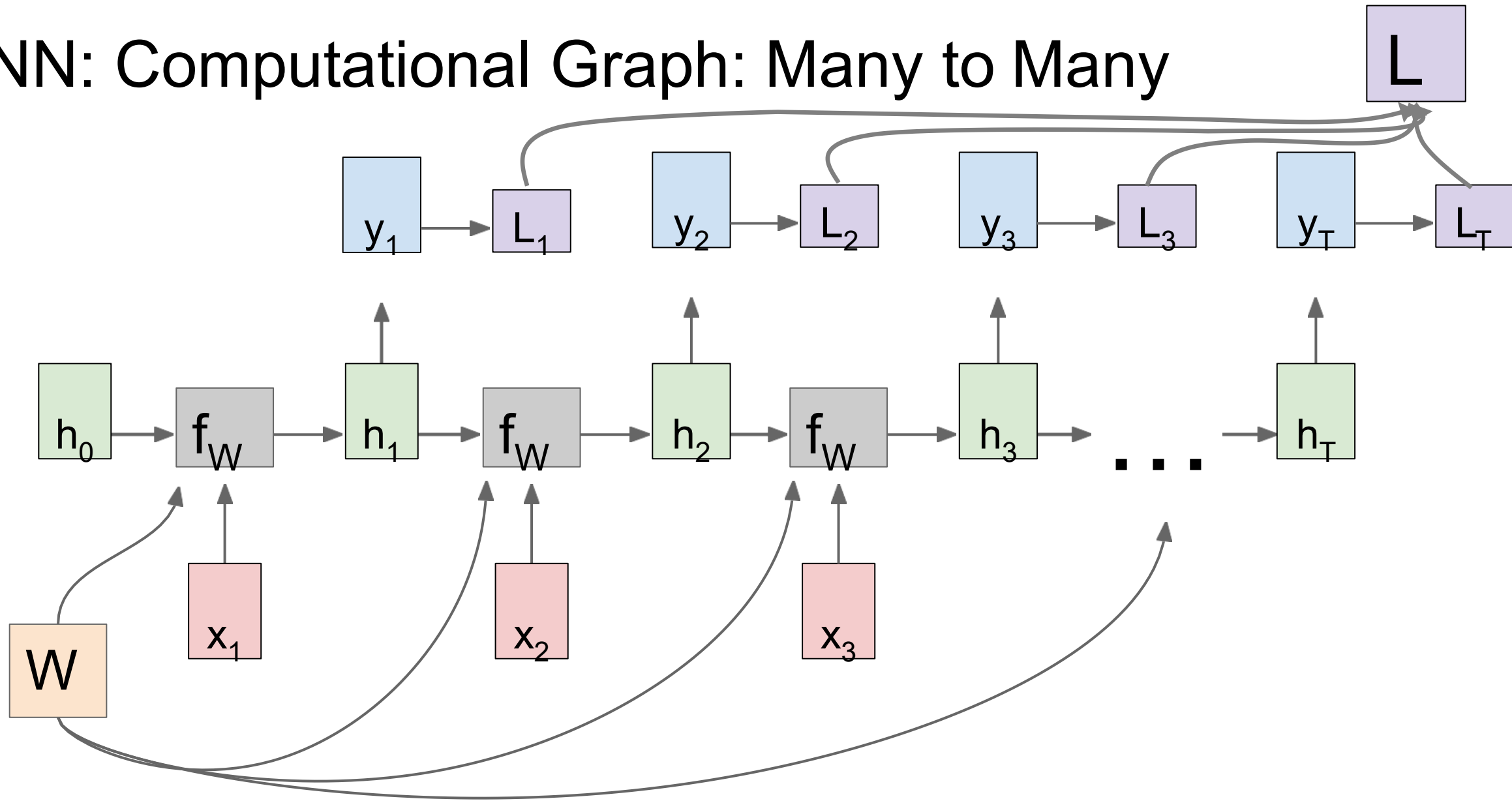
RNN: Computational Graph: Many to Many



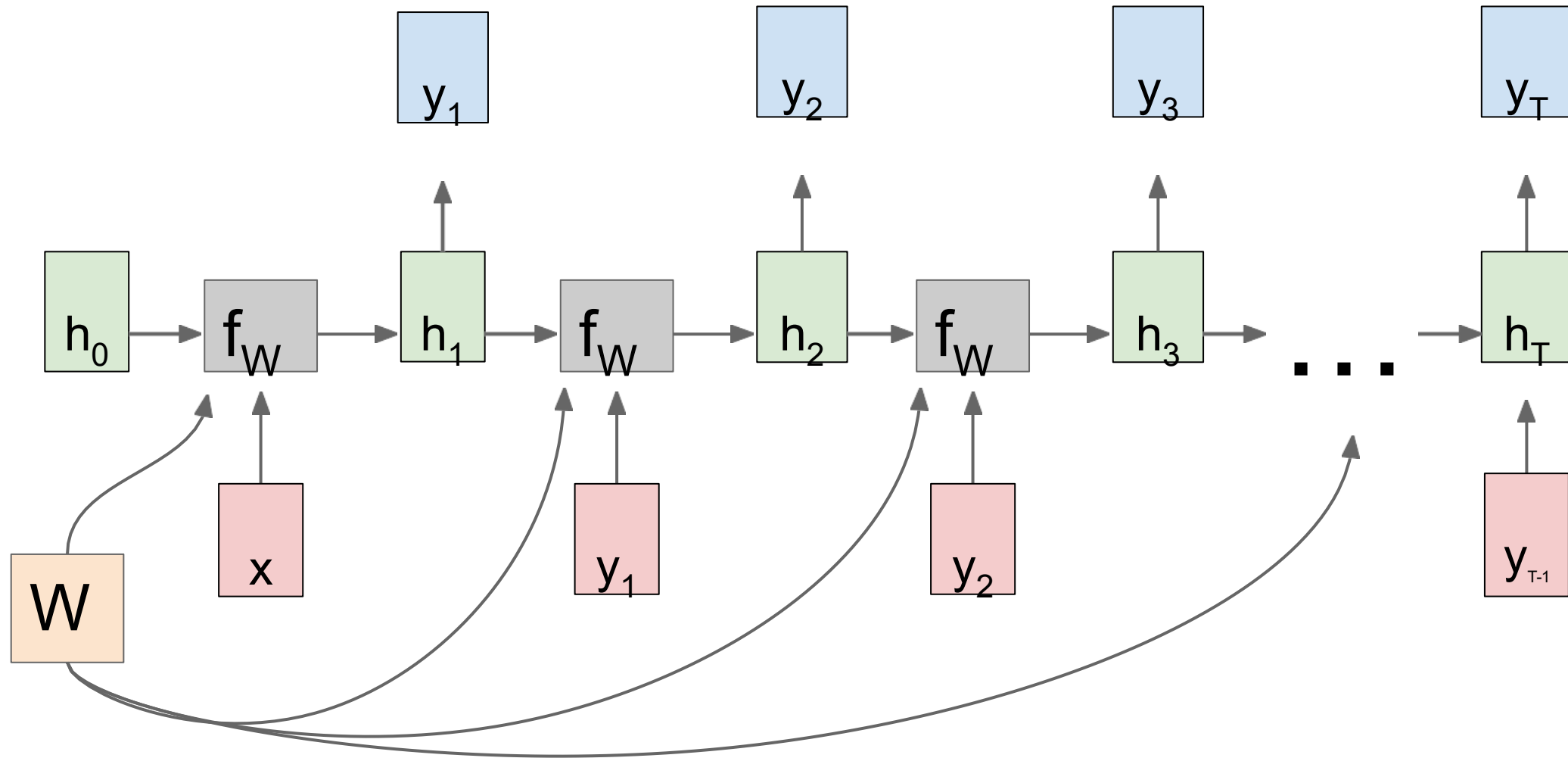
RNN: Computational Graph: Many to Many



RNN: Computational Graph: Many to Many

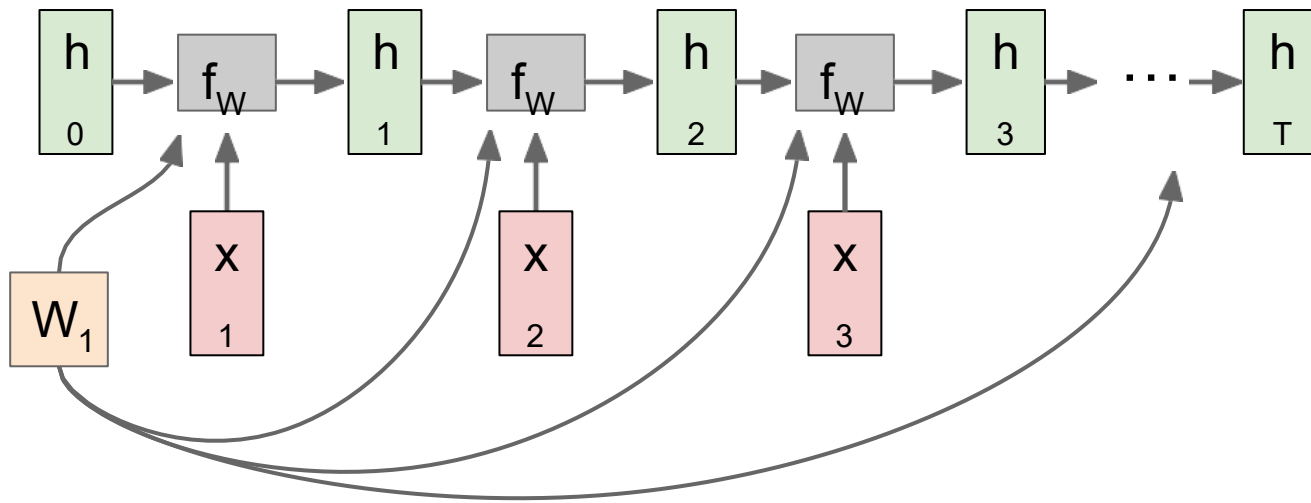


RNN: Computational Graph: One to Many



Sequence to Sequence: Many-to-one + one-to-many

Many to one: Encode input sequence in a single vector

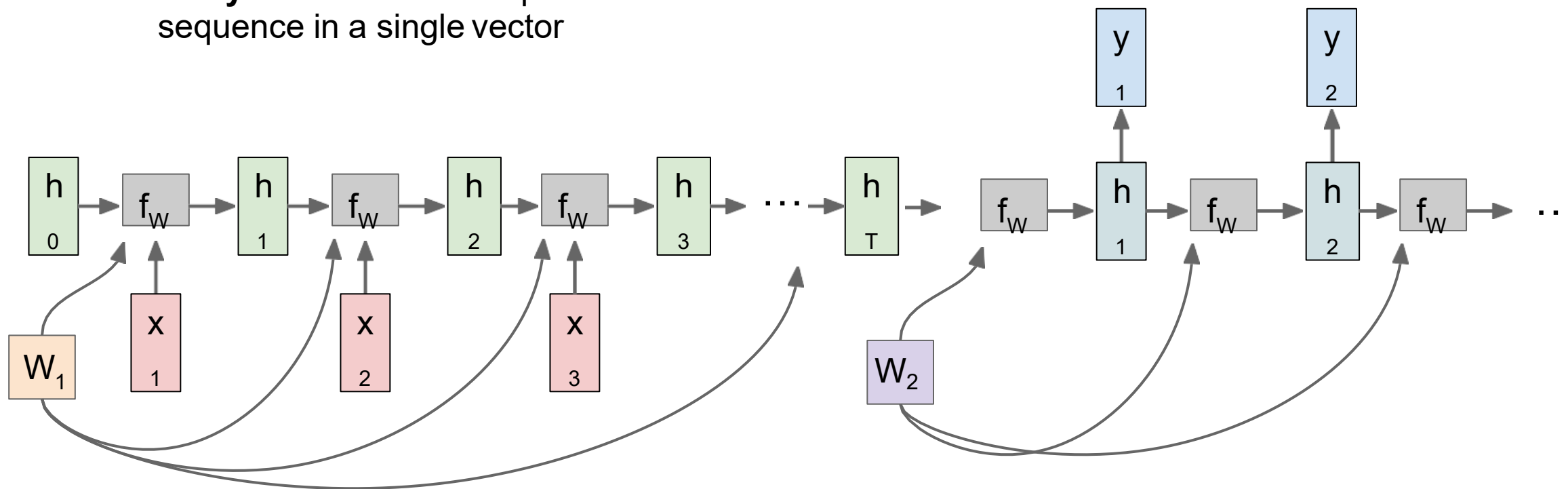


Sutskever et al, "Sequence to Sequence Learning with Neural Networks", NIPS 2014

Sequence to Sequence: Many-to-one + one-to-many

Many to one: Encode input sequence in a single vector

One to many: Produce output sequence from single input vector

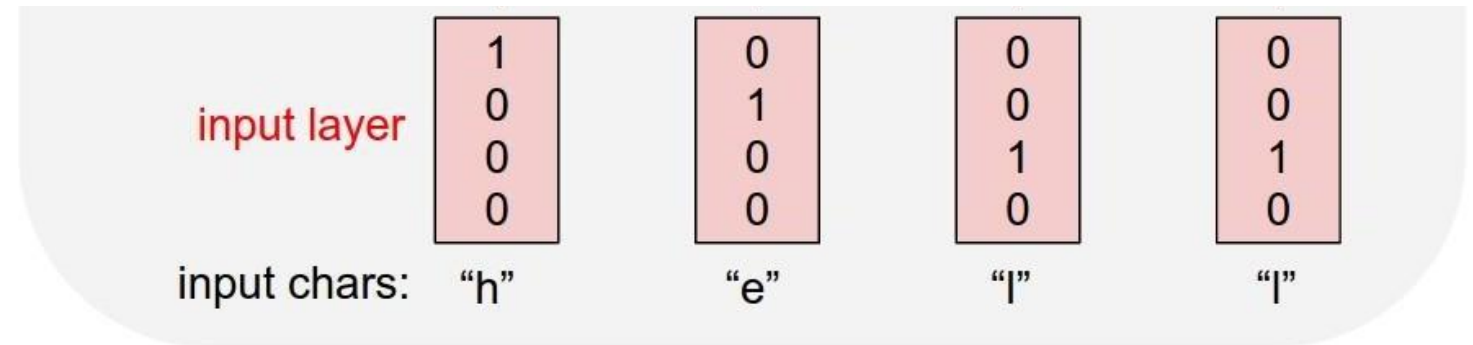


Sutskever et al, "Sequence to Sequence Learning with Neural Networks", NIPS 2014

Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

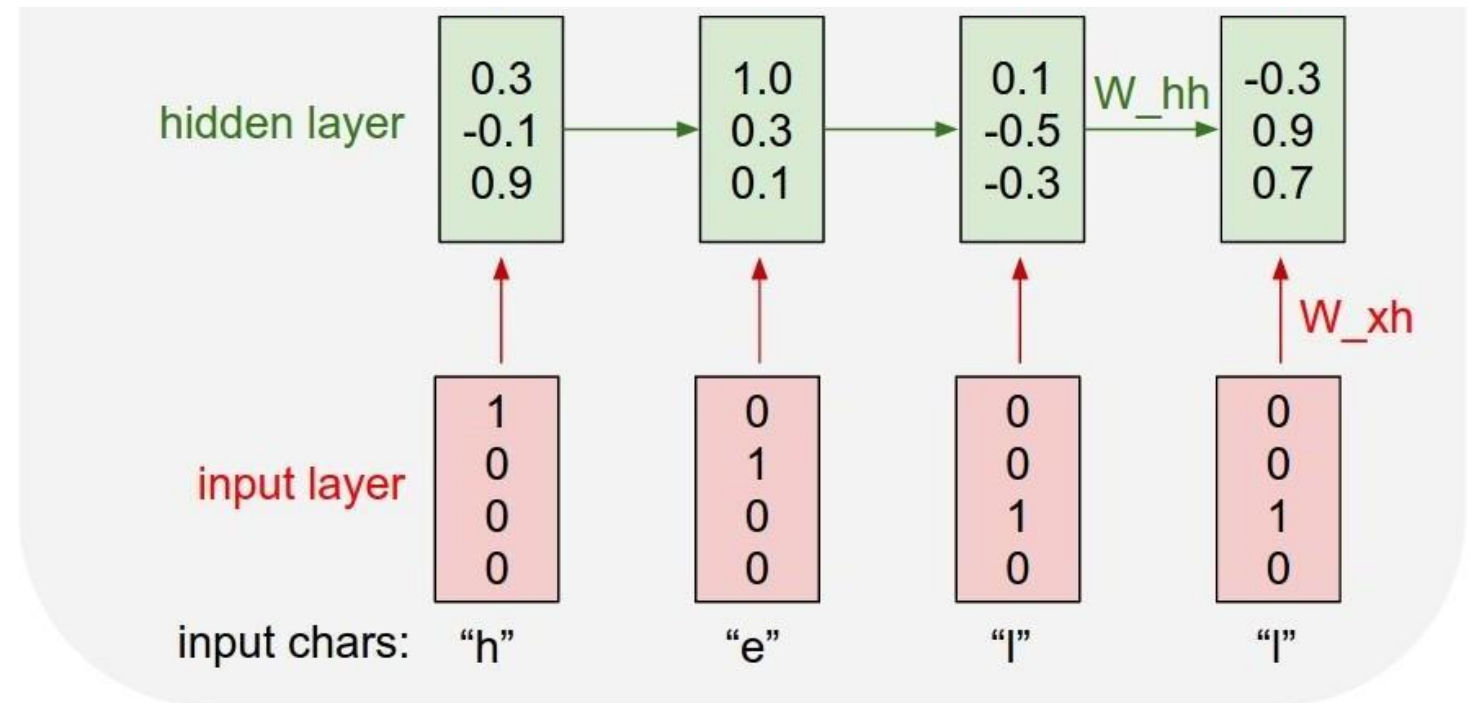


Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

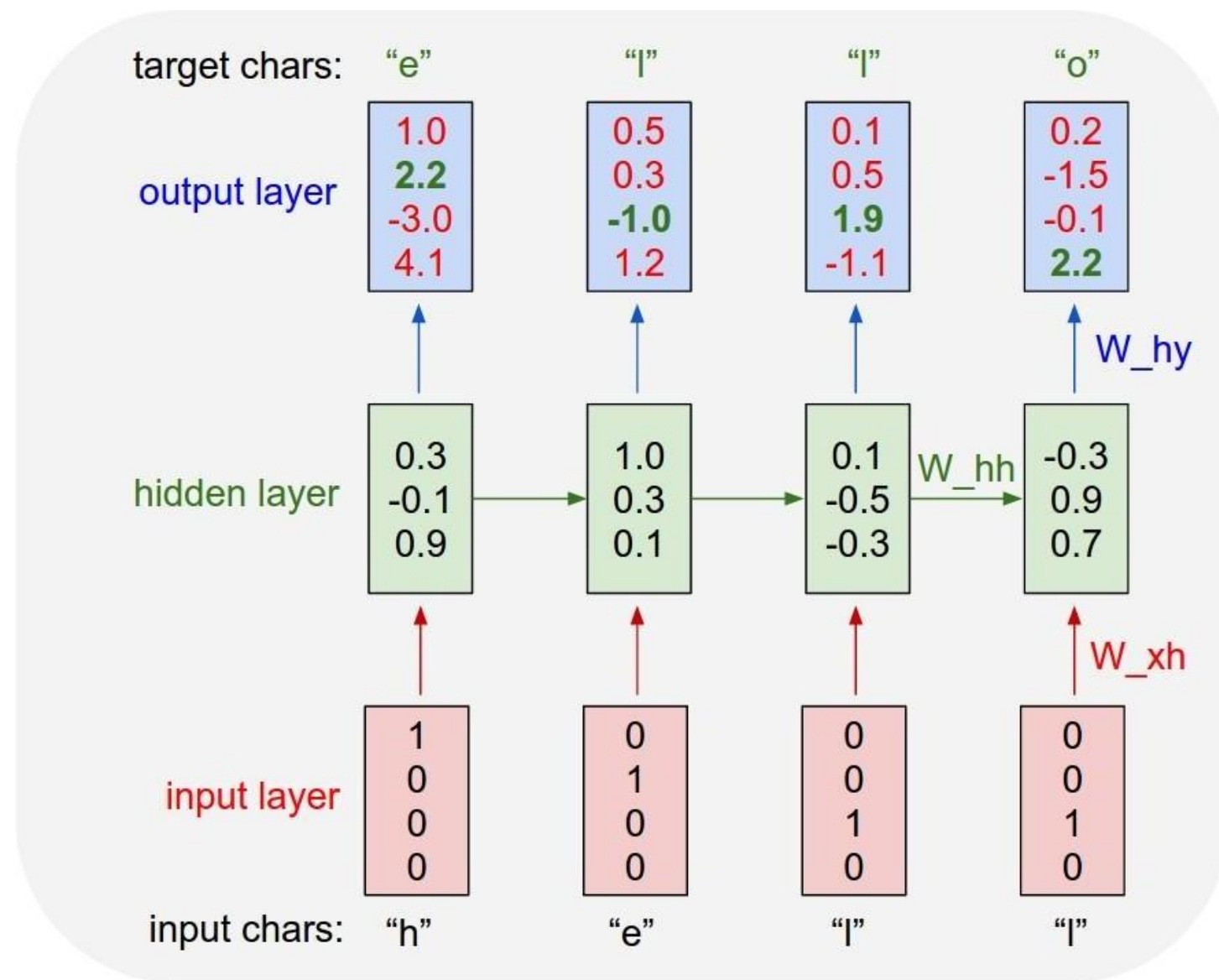


Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

aka GT forcing

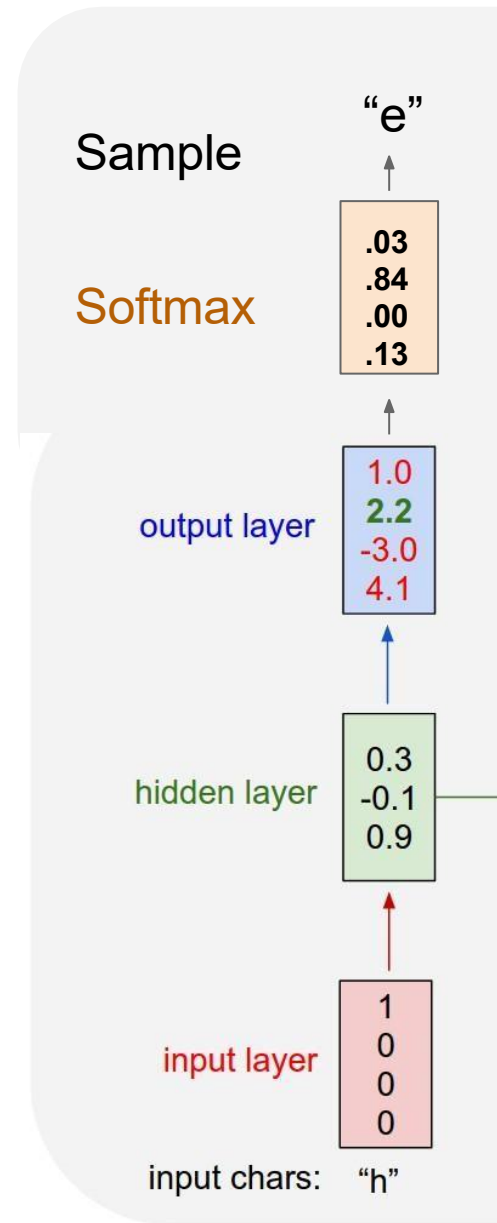


Exposure bias: He Zhang Zhou Glass ICLR 2020 Quantifying Exposure Bias for Neural Language Generation

Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

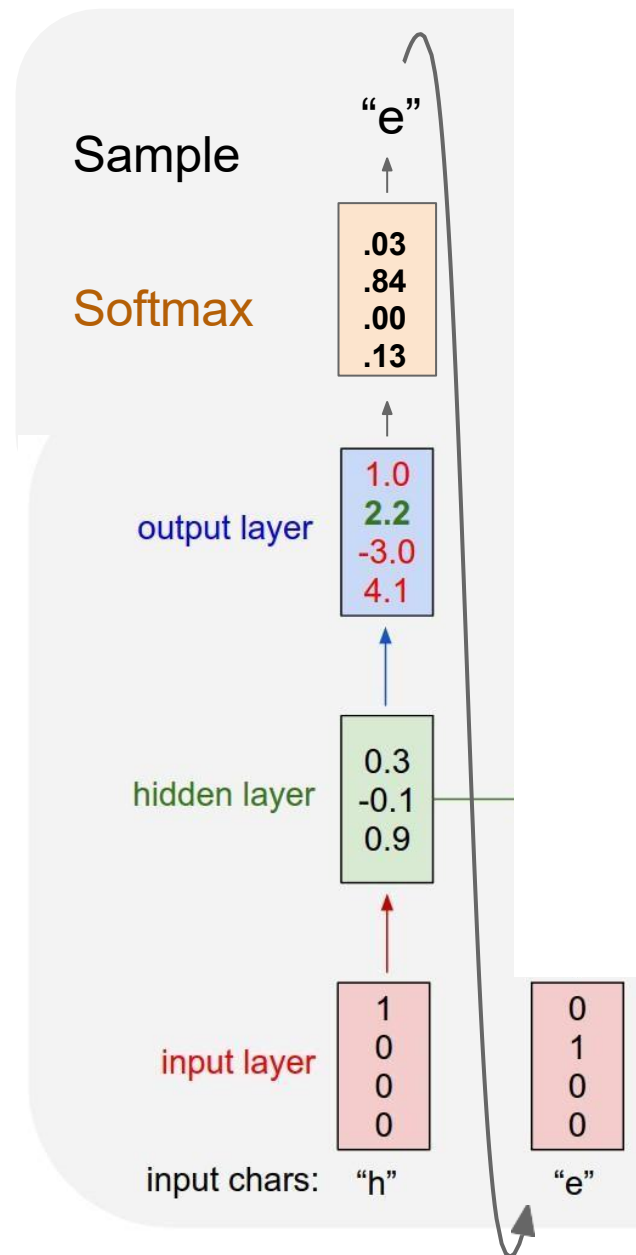
At test-time sample
characters one at a time,
feed back to model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

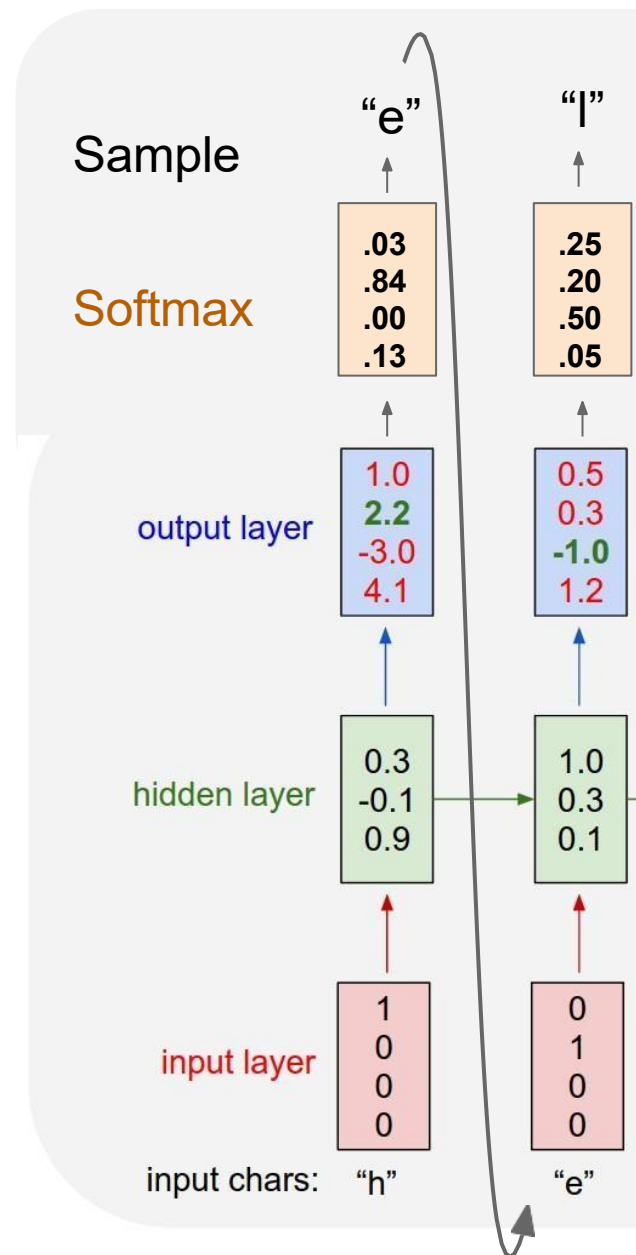
At test-time sample
characters one at a time,
feed back to model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

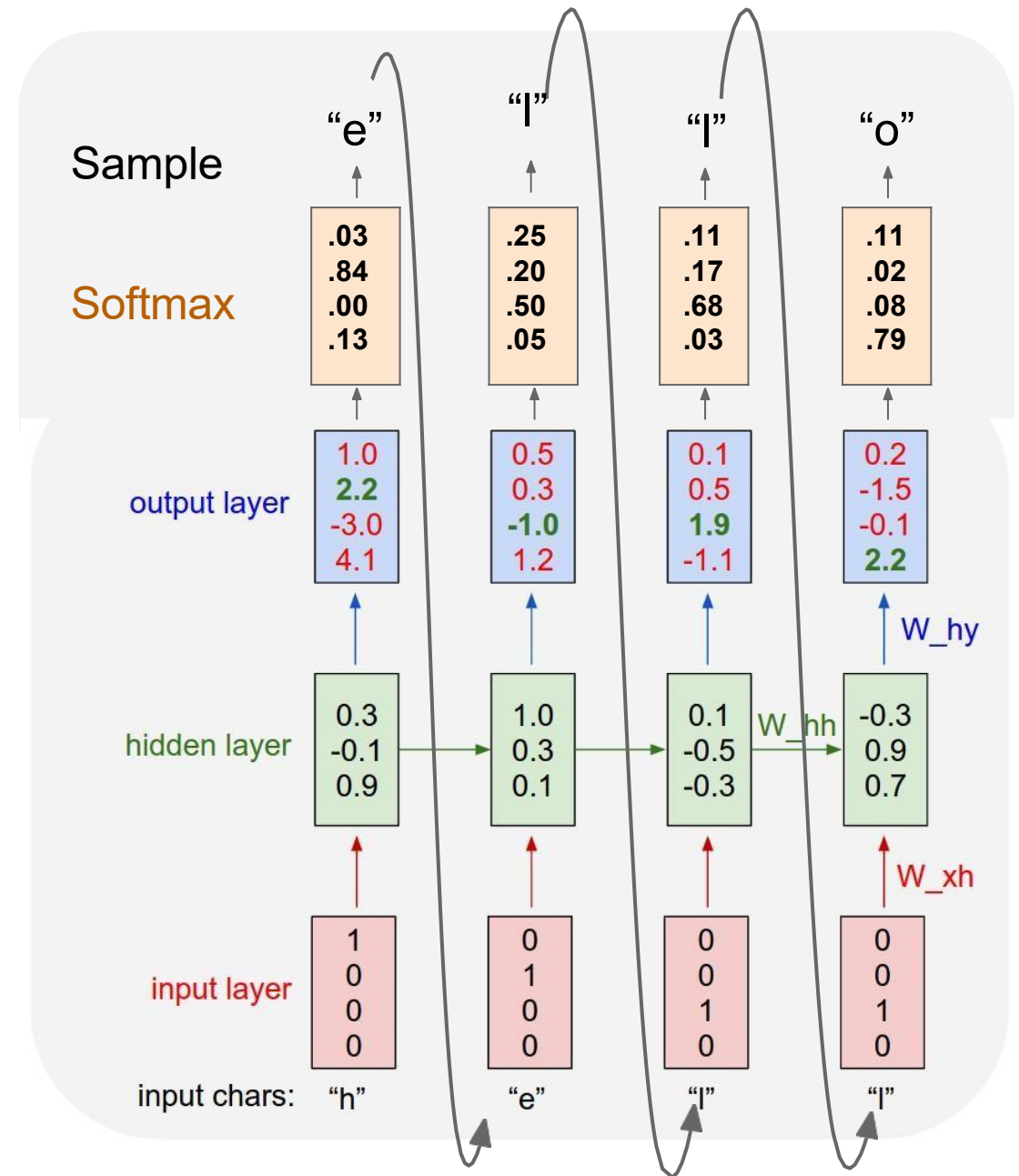
At test-time sample
characters one at a time,
feed back to model



Example: Character-level Language Model Sampling

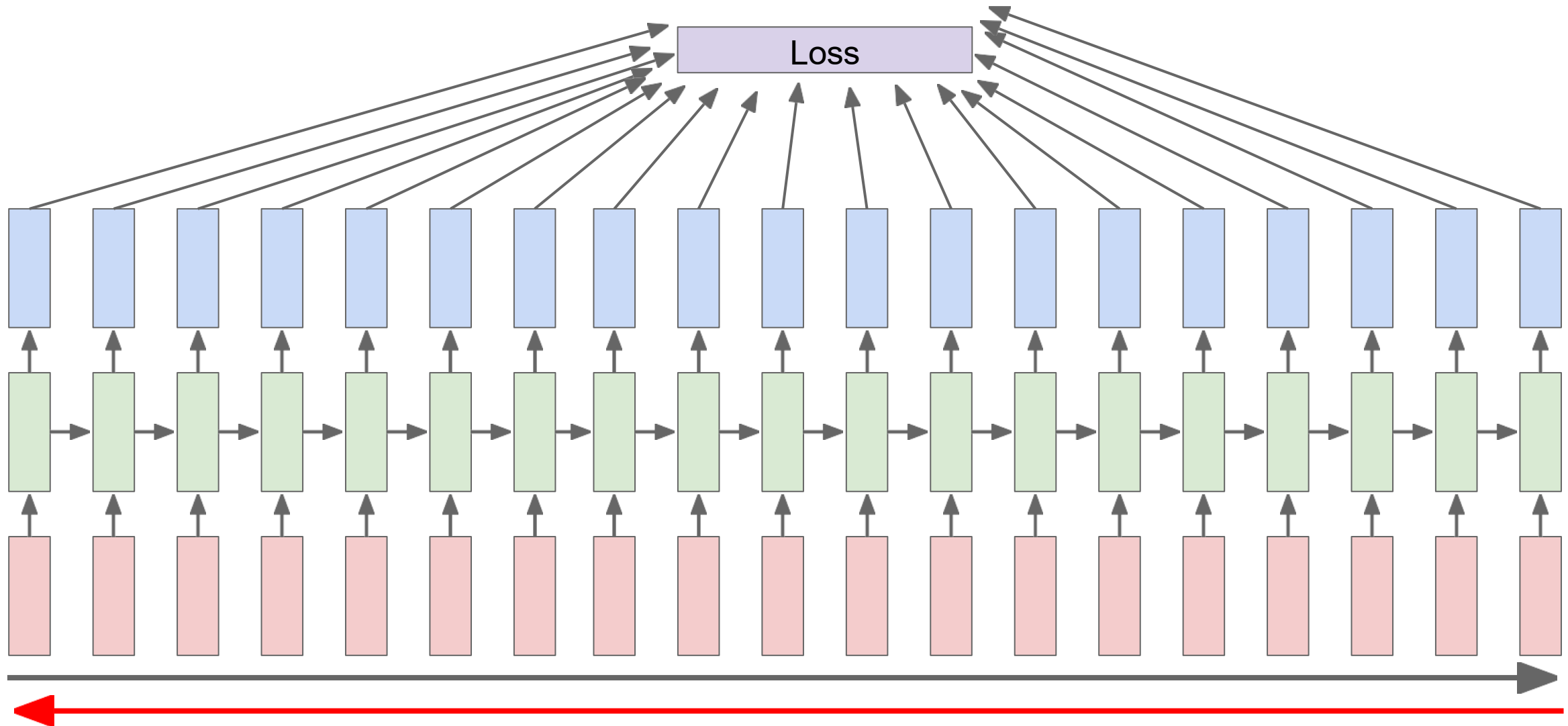
Vocabulary:
[h,e,l,o]

At test-time sample
characters one at a time,
feed back to model



Backpropagation through time

Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient



Suppose we had the training sentence “cat sat on mat”

We want to train a **language model**:

$P(\text{next word} \mid \text{previous words})$

i.e. want these to be high:

$P(\text{cat} \mid [<S>])$

$P(\text{sat} \mid [<S>, \text{cat}])$

$P(\text{on} \mid [<S>, \text{cat}, \text{sat}])$

$P(\text{mat} \mid [<S>, \text{cat}, \text{sat}, \text{on}])$

$P(<E> \mid [<S>, \text{cat}, \text{sat}, \text{on}, \text{mat}])$

Image Captioning

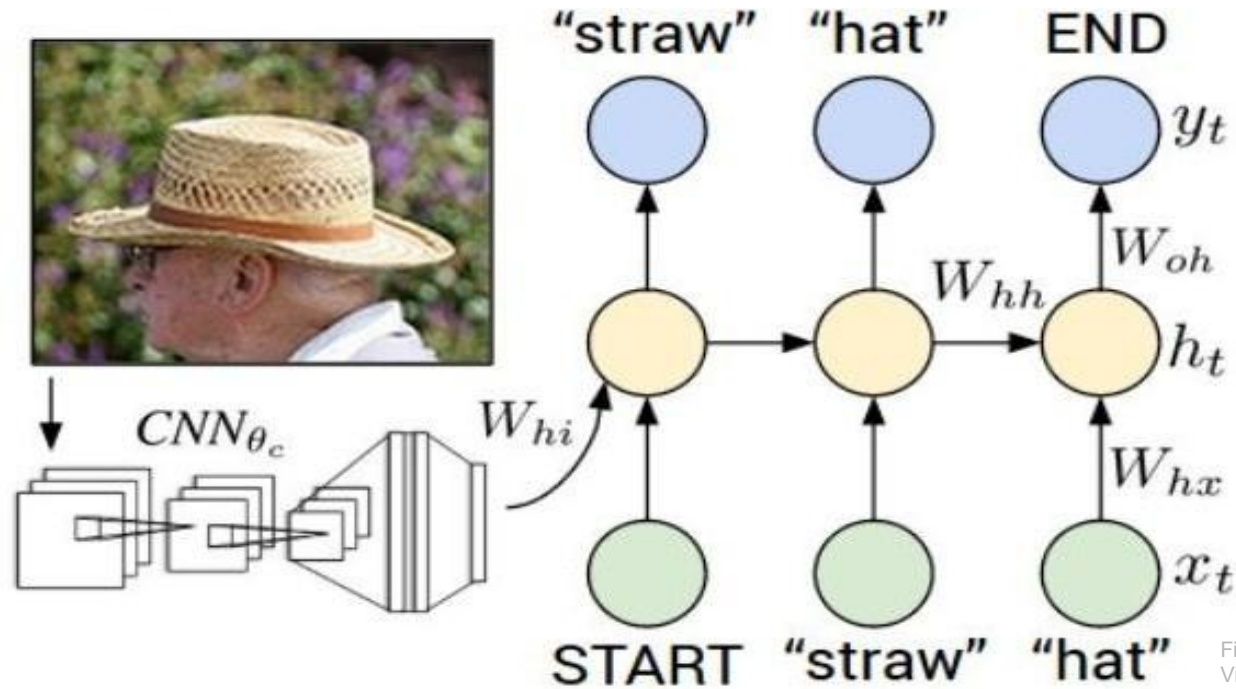


Figure from Karpathy et al, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015; figure copyright IEEE, 2015. Reproduced for educational purposes.

Explain Images with Multimodal Recurrent Neural Networks, Mao et al.

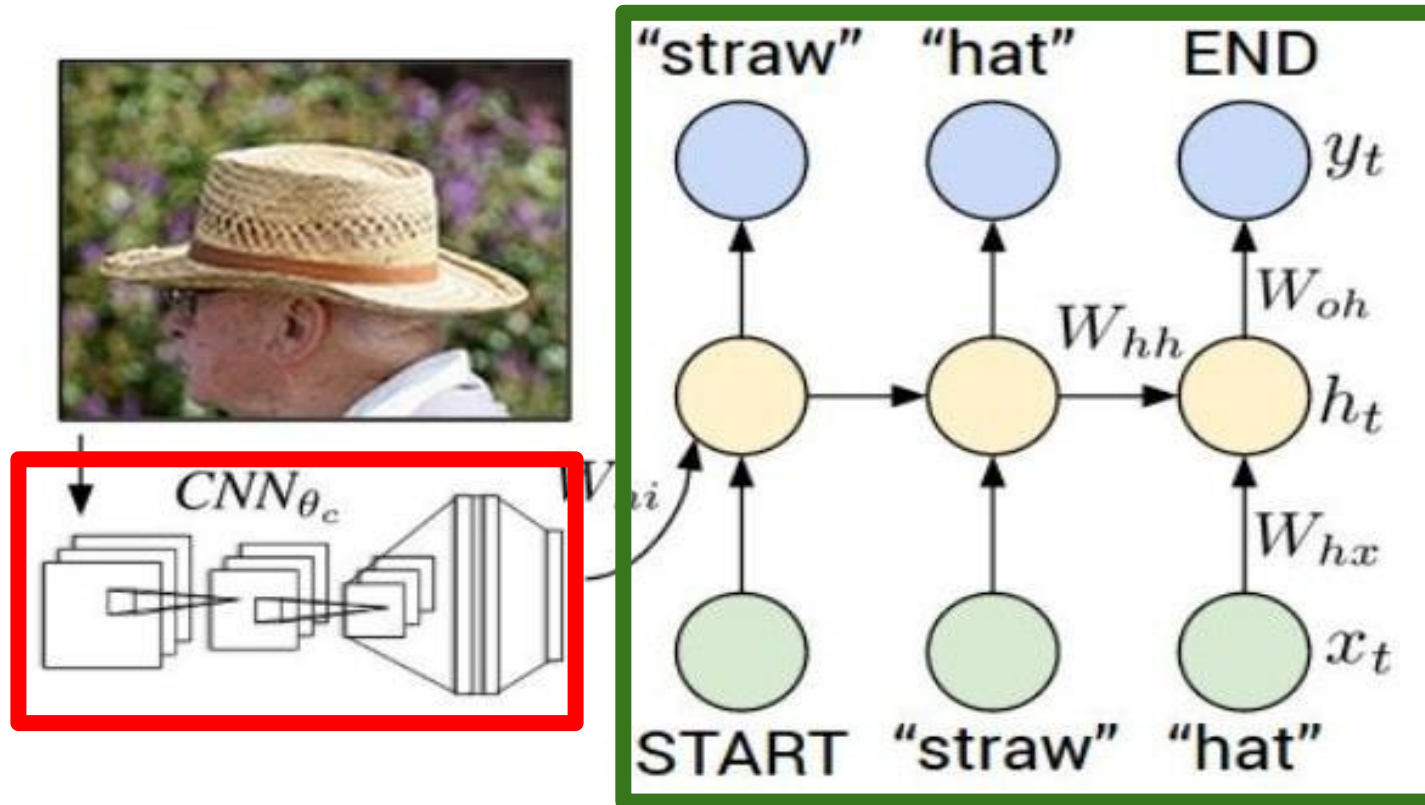
Deep Visual-Semantic Alignments for Generating Image Descriptions, Karpathy and Fei-Fei

Show and Tell: A Neural Image Caption Generator, Vinyals et al.

Long-term Recurrent Convolutional Networks for Visual Recognition and Description, Donahue et al.

Learning a Recurrent Visual Representation for Image Caption Generation, Chen and Zitnick

Recurrent Neural Network



Convolutional Neural Network



test image

This image is CC0 public domain

image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

maxpool

FC-4096

FC-4096

FC-1000

softmax



test image

image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

maxpool

FC-4096

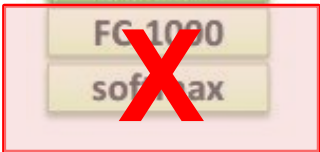
FC-4096

FC-1000

softmax



test image



image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

maxpool

FC-4096

FC-4096



test image

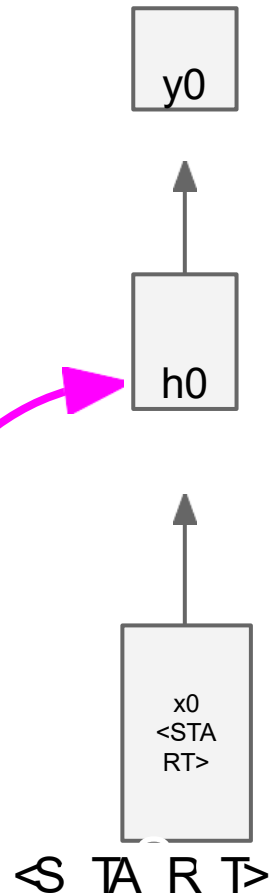
x0
<STA
RT>

<S T A R T>



V

W_{ih}



test image

before:

$$h = \tanh(W_{xh} * x + W_{hh} * h)$$

now:

$$h = \tanh(W_{xh} * x + W_{hh} * h + W_{ih} * v)$$

image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

maxpool

FC-4096

FC-4096



test image

y0

h0

x0
<STA
RT>

straw

sample!

<S TA R T>

image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

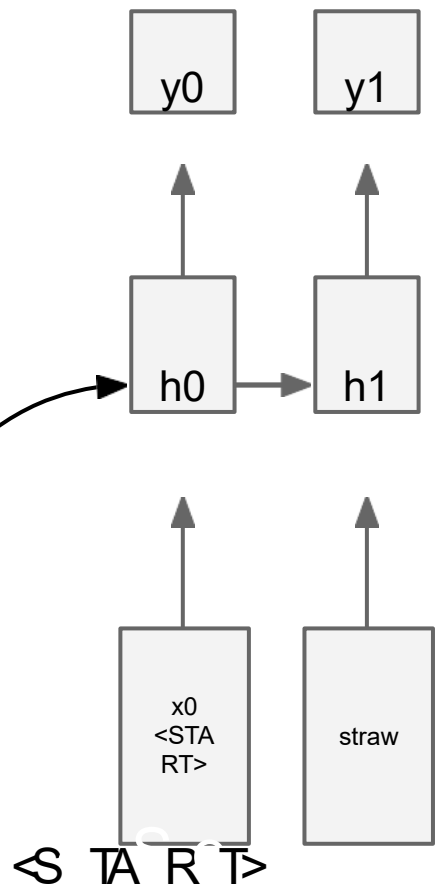
maxpool

FC-4096

FC-4096



test image



image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

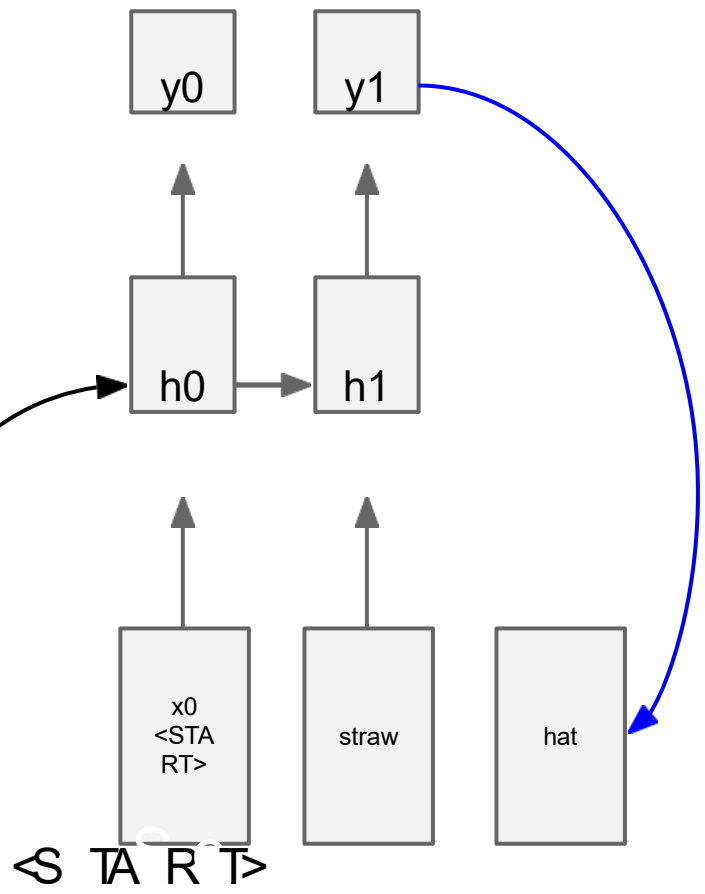
maxpool

FC-4096

FC-4096



test image



image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

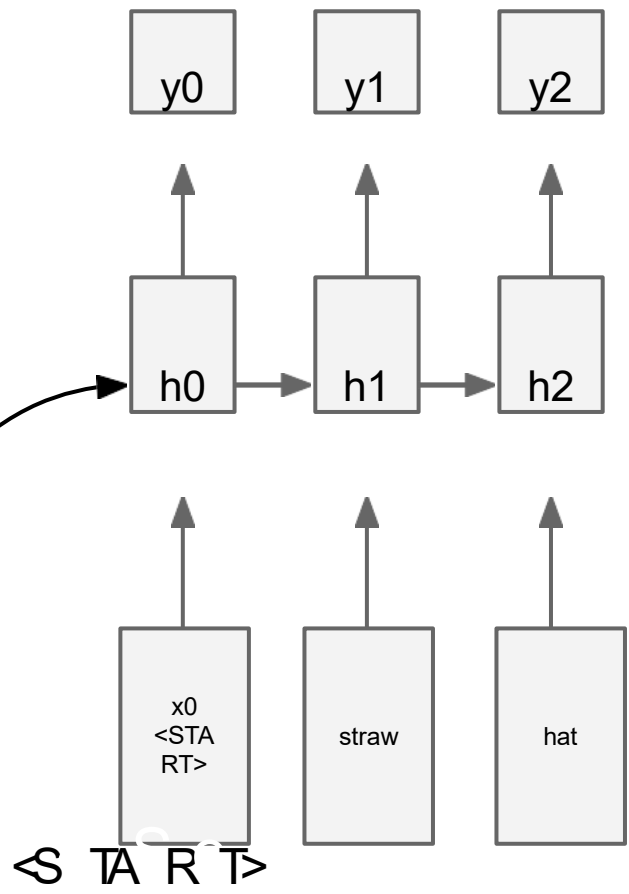
maxpool

FC-4096

FC-4096



test image



image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

maxpool

FC-4096

FC-4096



test image

y0 y1 y2

h0 h1 h2

x0
<START>

straw

hat

<START>

sample
<END> token
=> finish.

Image Captioning: Example Results

Captions generated using [neuraltalk2](#)
All images are [CC0 Public domain](#):
[cat suitcase](#), [cat tree](#), [dog](#), [bear](#),
[surfers](#), [tennis](#), [giraffe](#), [motorcycle](#)



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court



Two giraffes standing in a grassy field



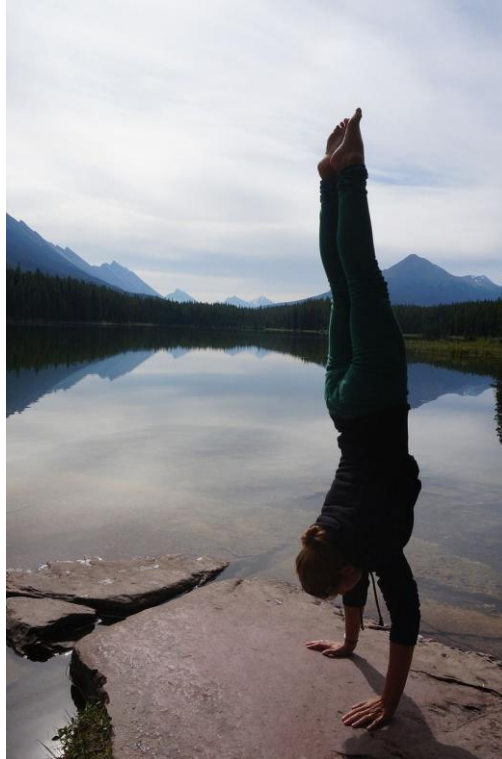
A man riding a dirt bike on a dirt track

Image Captioning: Failure Cases

Captions generated using [neuraltalk2](#)
All images are [CC0 Public domain](#): [fur coat](#), [handstand](#), [spider web](#), [baseball](#)



A woman is holding a cat in her hand



A woman standing on a beach holding a surfboard



A bird is perched on a tree branch

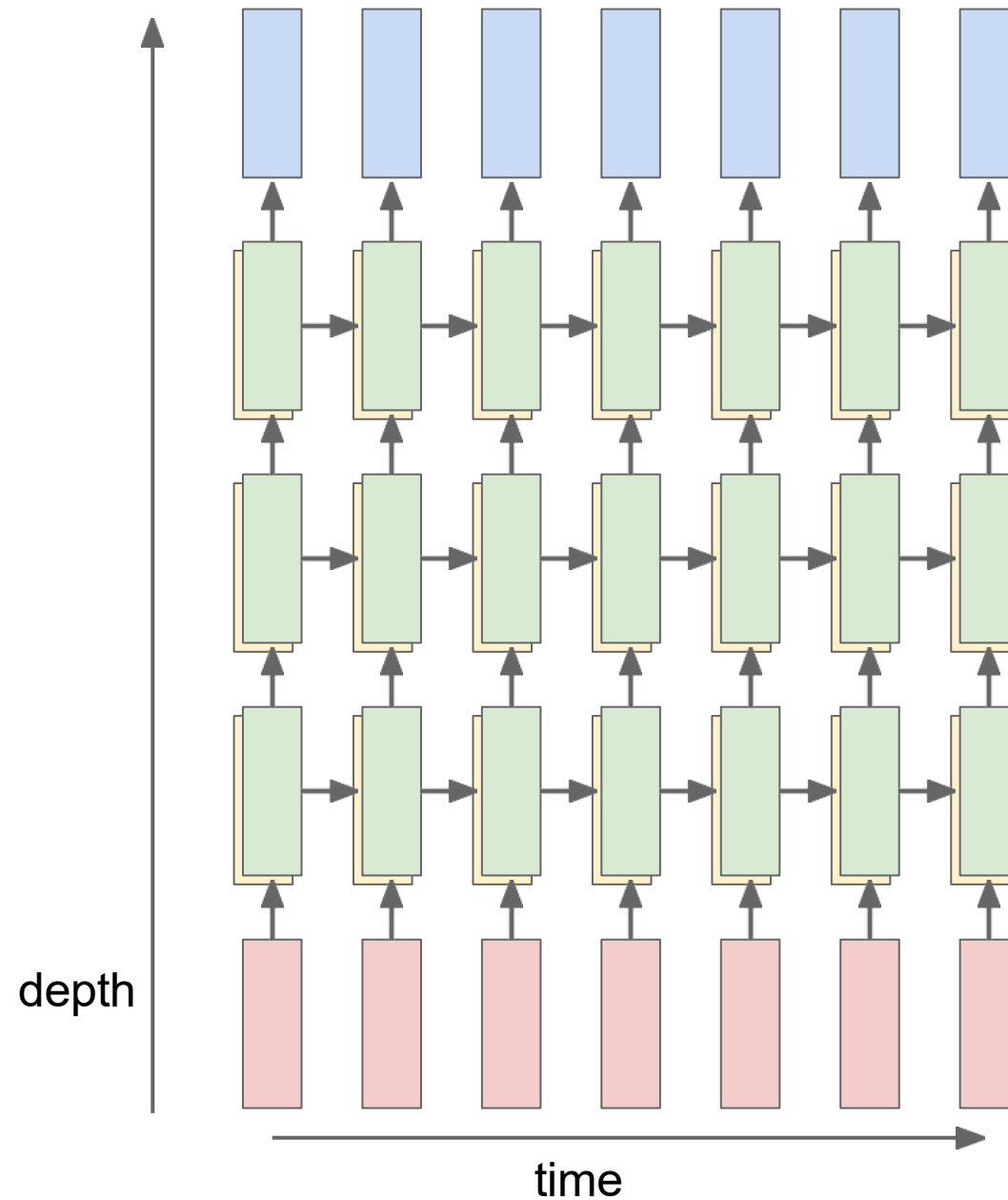


A person holding a computer mouse on a desk



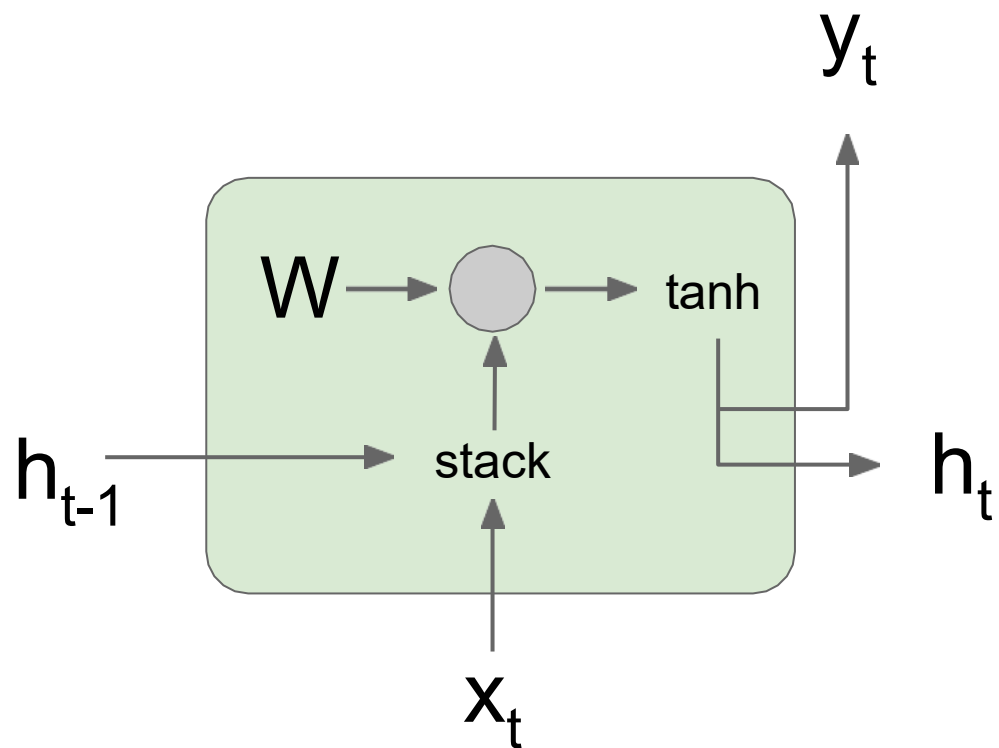
A man in a baseball uniform throwing a ball

Multilayer RNNs



Vanilla RNN Gradient Flow

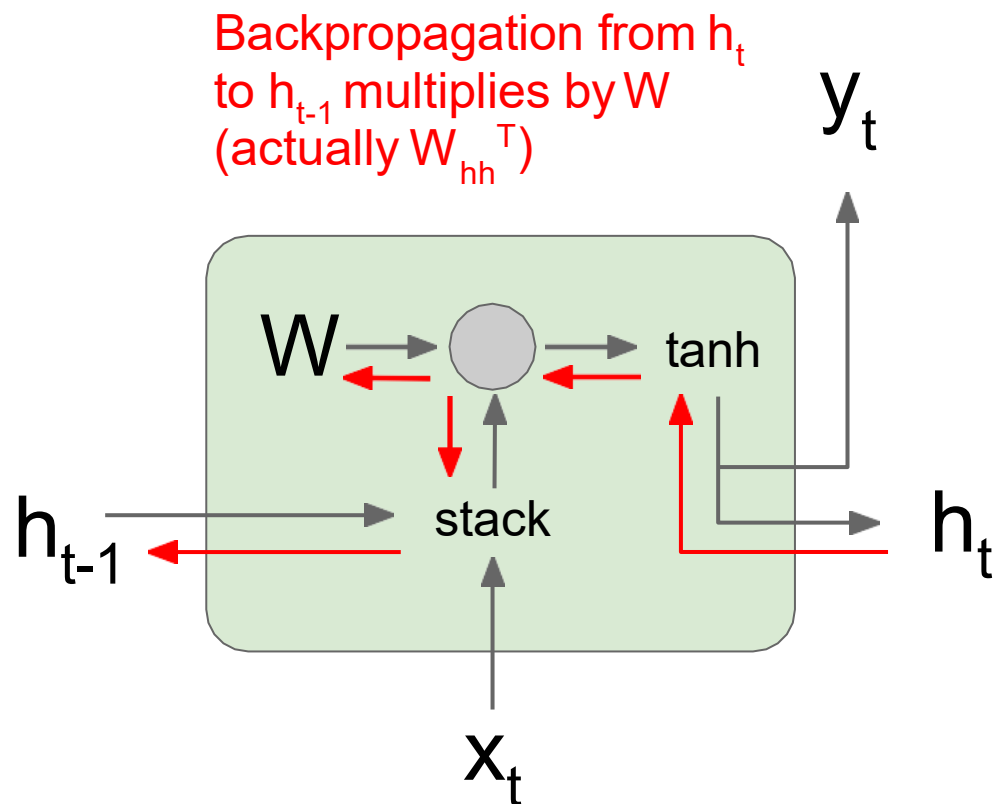
Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

Vanilla RNN Gradient Flow

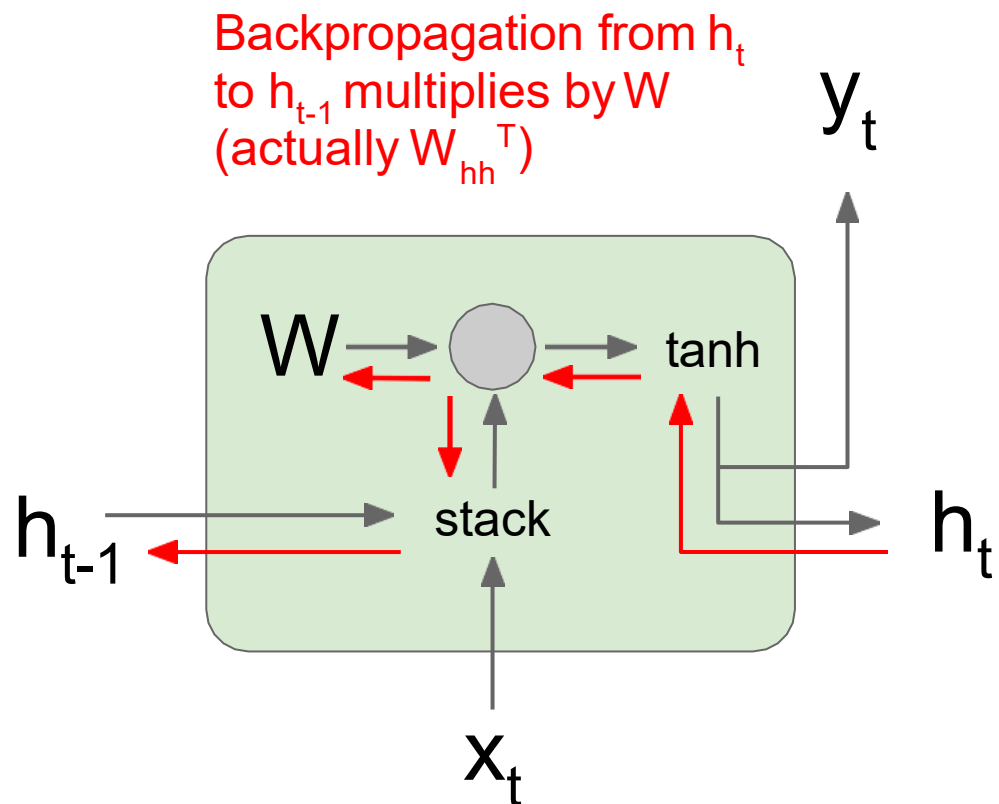
Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



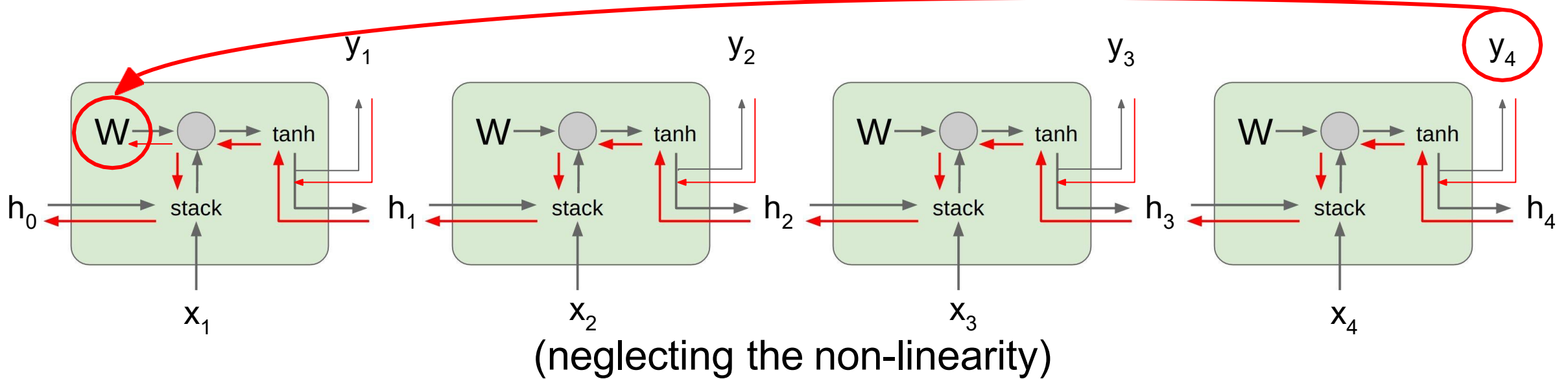
$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Largest singular value > 1 : $\rightarrow$ Gradient clipping
Exploding gradients

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W^{T-1}_{in}} \frac{\partial h_1}{\partial W}$$

Largest singular value < 1 : $\rightarrow$ Change RNN architecture
Vanishing gradients

RNN tradeoffs

RNN Advantages:

- Can process any length input
- Computation for step t can (in theory) use information from many steps back
- Model size doesn't increase for longer input
- Same weights applied on every timestep, so there is symmetry in how inputs are processed.

RNN Disadvantages:

- Recurrent computation is slow
- In practice, difficult to access information from many steps back

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$
$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

Four gates

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

Cell state

$$c_t = f \odot c_{t-1} + i \odot g$$

Hidden state

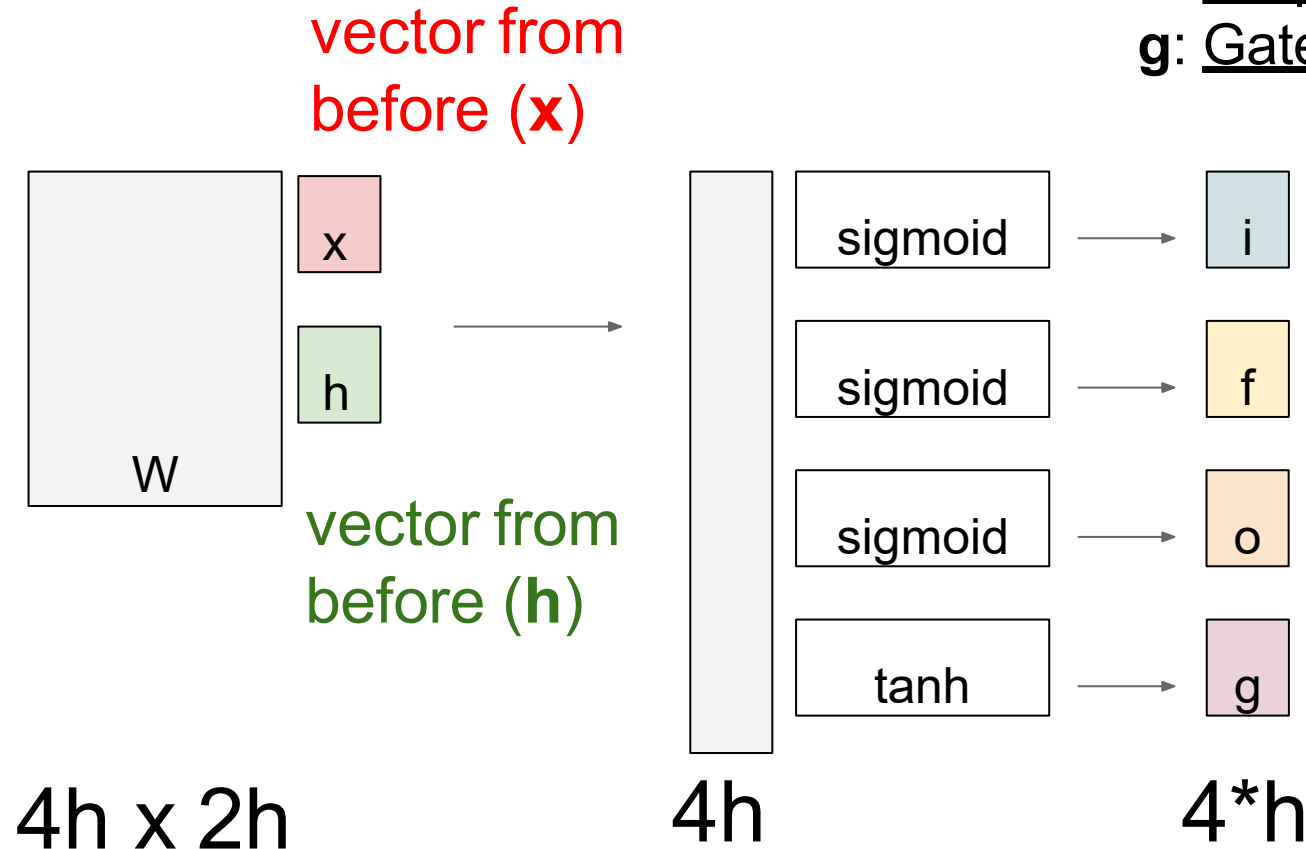
$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]

- i: Input gate, whether to write to cell
- f: Forget gate, Whether to erase cell
- o: Output gate, How much to reveal cell
- g: Gate gate (?), How much to write to cell



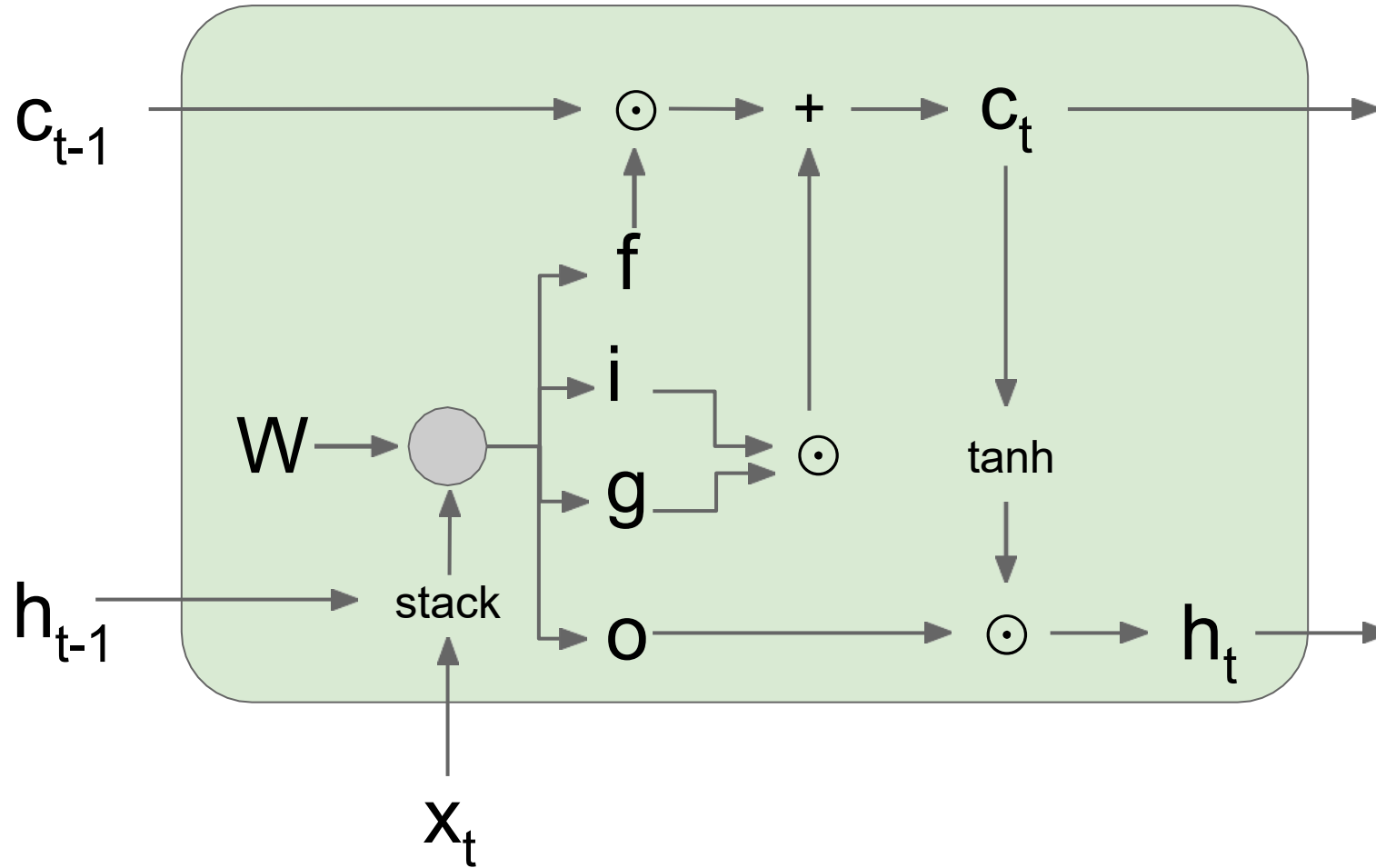
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



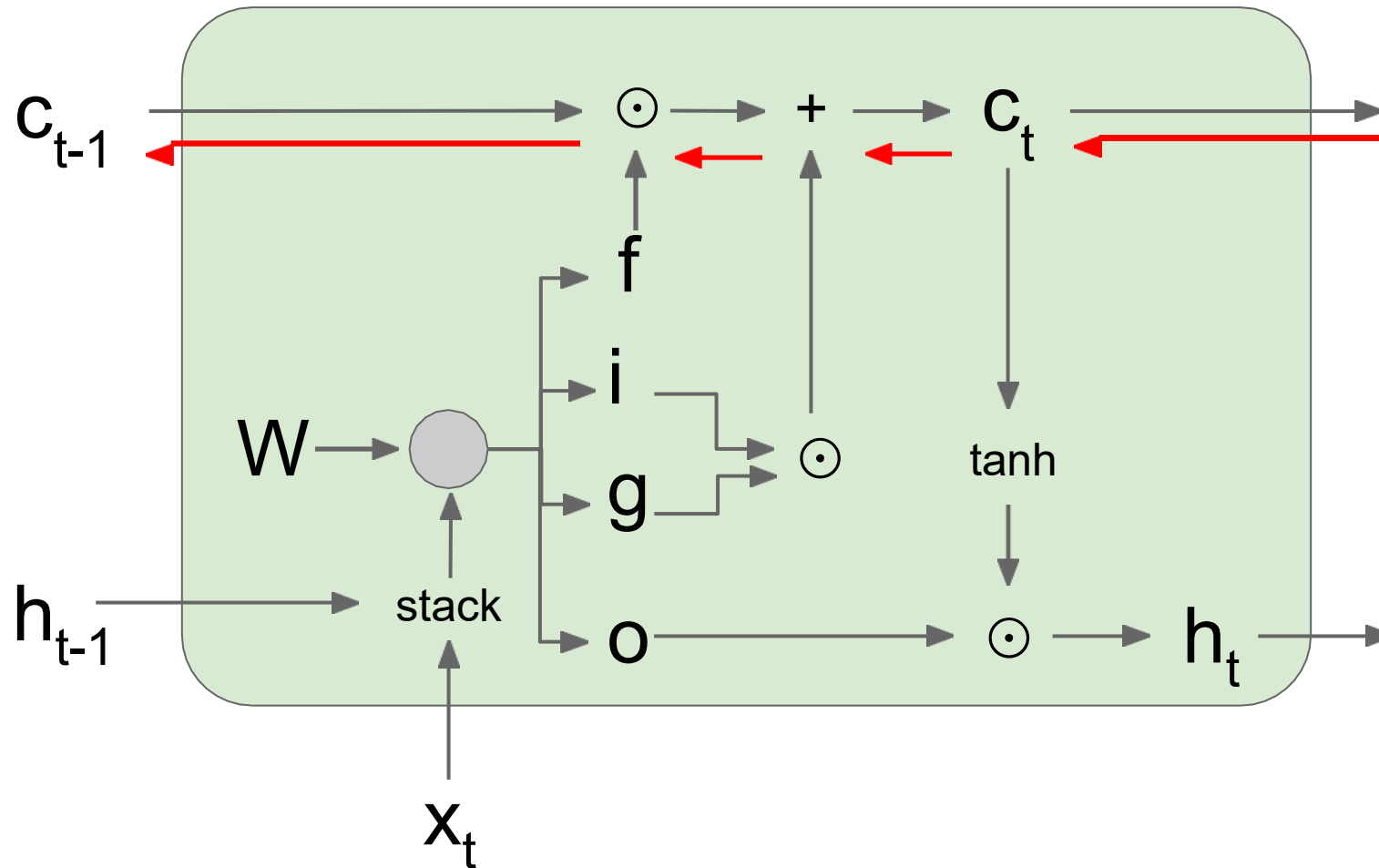
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM): Gradient Flow

[Hochreiter et al., 1997]



Backpropagation from c_t to c_{t-1} only elementwise multiplication by f , no matrix multiply by W

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

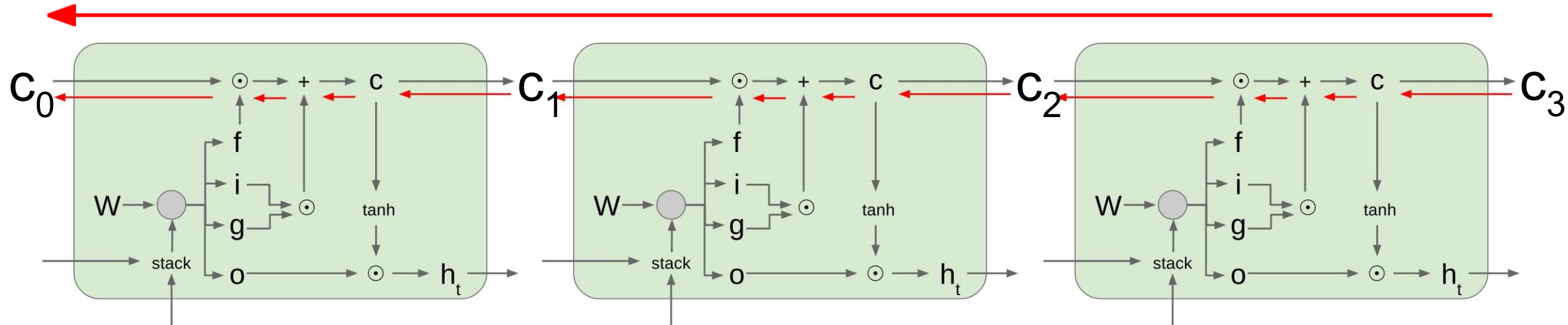
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM): Gradient Flow

[Hochreiter et al., 1997]

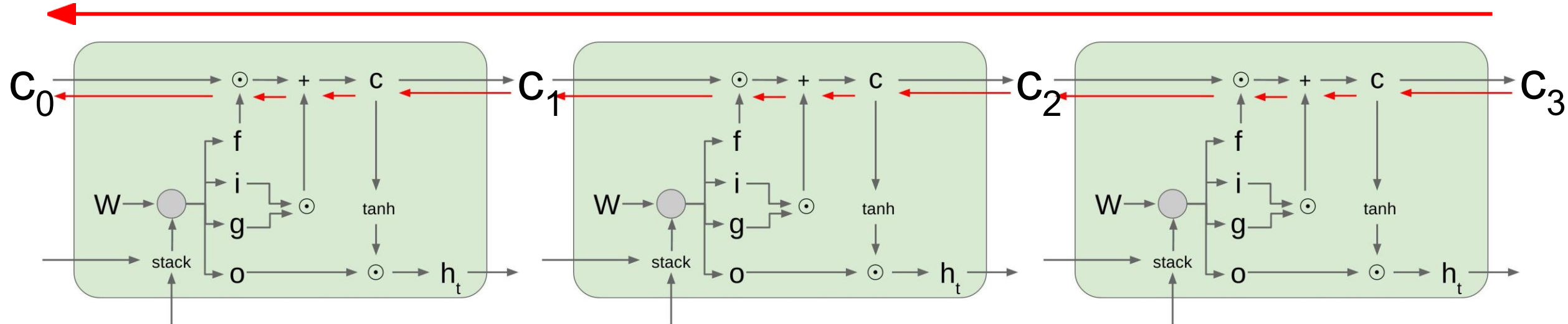
Uninterrupted gradient flow!



Long Short Term Memory (LSTM): Gradient Flow

[Hochreiter et al., 1997]

Uninterrupted gradient flow!



Notice that the gradient contains the f gate's vector of activations

- allows better control of gradients values, using suitable parameter updates of the forget gate.

Also notice that more control is added through the f , i , g , and o gates

- better balancing of gradient values

Summary

- RNNs allow a lot of flexibility in architecture design
- Vanilla RNNs are simple but don't work very well
- Common to use LSTM or GRU: their additive interactions improve gradient flow
- Backward flow of gradients in RNN can explode or vanish. Exploding is controlled with gradient clipping. Vanishing is controlled with additive interactions (LSTM)
- Better/simpler architectures are a hot topic of current research, as well as new paradigms for reasoning over sequences
- Better understanding (both theoretical and empirical) is needed.

Thank you

Acknowledges: some slides and material from Fei-Fei Li, Justin Johnson, Serena Yeung, Jeff Donahue, Irtiza Hasan, Marco Cristani, Ashish Vaswani, Anna Huang, Jacob Devlin, Jay Alammar



SAPIENZA
UNIVERSITÀ DI ROMA